

Resonant Symbolic Operator Calculus (RSOC)

Formal Convergence Proof of $\chi(t)$ Under Adversarial Phase Conditions
including Lemma 5: One-Step Simultaneous Detection

AI.Web Inc. · Nicholas Bogaert

Version 4.8 · March 2026 · **Reframed for cs.NE/cs.LG: Introduction, S1–S9, failure-mode framing, abstract**

Abstract

Problem. Deep recursive AI systems can exhibit categorical phase-skip failure modes that evade statistical drift detectors entirely. Classical sequential change-point procedures (CUSUM, GLR, Page’s test) are vulnerable to distribution-matching evasion: an adversary who knows the detector’s sufficient statistic can suppress it indefinitely by steering the post-change distribution toward the pre-change distribution [1, 10]. For a *categorical* skip — a system jumping from phase state S_5 to S_8 in a single step, bypassing S_6 and S_7 entirely — no distribution to match exists. The skip is its own fingerprint. Yet no formally proved monitor exploits this structure.

Contribution. We prove a convergence guarantee for $\chi(t)$, an event-triggered corrective controller in the Resonant Symbolic Operator Calculus (RSOC): a **hybrid discrete/continuous** monitor for K -phase recursive AI processes. The monitored system has a finite discrete phase state $\Phi_k \in \{S_1, \dots, S_K\}$ ($K = 9$ in the reference implementation), a continuous structural drift metric $\varepsilon_s(t) \geq 0$, and a corrective controller $\chi(t)$ that fires on a guard condition. The key detection mechanism — d_{score} — is a *deterministic categorical predicate*, not a statistical estimator. For the single-step $S_5 \rightarrow S_8$ phase-skip failure mode, it is structurally resistant to the distribution-matching evasion strategy that defeats statistical detectors [1, 8], because there is no distribution to match: the predicate checks discrete state identity, not distributional proximity.¹

Main result. Under Assumption A1 (Trusted Initialization) and $\beta > P_{\max}/\varrho_{\min}$, $\chi(t)$ converges before ε_s reaches the hard ceiling $\theta_{\text{hard}} = 0.8$ at all recursion depths (Theorem 1). The convergence condition is met analytically by the reference implementation: $\beta_{\text{impl}}/f_{144} \geq \ln 5 \approx 1.609 > \frac{1}{2} = P_{\max}^{(\theta_w)}/\varrho_{\min}$ (Proposition 6).

Scope. Theorem 1 covers the single-step phase-skip adversary. A multi-step acceleration variant ($S_5 \rightarrow S_6 \rightarrow S_7 \rightarrow S_8$, each step individually valid) is handled by an implemented phase velocity accumulator $v(t)$; formal proof deferred (OQ3). Baseline integrity rests on Assumption A1, hardened via $N \geq 3$ consensus capture and HMAC anchor.

¹Internally this failure mode is called a “phase-jump attack”; we use that term throughout. The reference implementation names the nine states descriptively; this paper uses S_1 – S_9 .

Introduction

Recursive AI systems — recurrent networks, multi-agent loops, deep inference pipelines — can fail through a structural mechanism that statistical monitors are ill-suited to catch: a *categorical phase skip*, in which the system jumps over one or more required processing stages in a single step. Unlike distributional drift, which changes the statistics of system outputs gradually, a categorical skip changes the *type* of the transition: the system goes from state S_5 directly to S_8 , bypassing S_6 and S_7 entirely. From a statistical standpoint this may look unremarkable — if the output distribution at S_8 is similar to what was expected, a likelihood-ratio test will not fire. The architectural constraint has been violated; the statistical signal is absent.

This gap is not incidental. It reflects a structural property of statistical detectors: they operate on distributional proximity, and an adversary (or a failing system) that produces outputs whose distribution is close to the expected distribution will evade them indefinitely [1, 10]. The evasion is not a matter of detector tuning; it is a consequence of what statistical detectors fundamentally measure. Hinton’s forward-forward algorithm [5] and related work on architectural alternatives to backpropagation-driven systems have sharpened the question of what *structural* guarantees are achievable in deep recursive systems — as distinct from the statistical approximations that dominate current monitoring practice.

This paper provides one such guarantee. We prove that a deterministic categorical predicate d_{score} , defined on the discrete state pair $(\Phi_{\text{prev}}, \Phi_{\text{act}})$, detects the single-step $S_5 \rightarrow S_8$ phase-skip with zero causal lag and no distributional parameters to evade (Lemma 5). We then prove that the event-triggered corrective controller $\chi(t)$ converges before the structural drift metric $\varepsilon_s(t)$ reaches its hard ceiling $\theta_{\text{hard}} = 0.8$, for all recursion depths $\rho \geq 1$, under an analytically verified condition on the recovery rate β (Theorem 1, Proposition 6). The proof maps directly onto hybrid systems and event-triggered control theory [12–14]; the detection predicate maps onto the guard condition of a hybrid automaton. No statistical assumptions are required.

The paper is organized as follows. Section 0 (System Classification) formally declares the hybrid system structure. Sections 1–5 establish notation, definitions, core formulae, and the RSOC–PCI motivational bridge. Sections 6–8 contain the lemmas, main theorem, and detection path table. Section 9 states open questions. Appendix A provides an empirical illustration of $\varepsilon_s(t)$ and $\chi(t)$ trajectories under attack vs. normal operation.

System Classification and Mathematical Framework

Remark 1 (Formal System Classification). *The monitoring system described in the Introduction is formally classified as a **hybrid discrete/continuous system** [13]. Concretely:*

- **Discrete layer.** *The process phase $\Phi_k \in \{1, \dots, K\}$ (here $K = 9$) evolves as*

a finite-state machine with deterministic transition rules. Phase changes are discrete events. The attack indicator d_{score} is a binary predicate on the current discrete state.

- **Continuous layer.** The drift functional $\varepsilon_s(t) \geq 0$ evolves quasi-continuously between discrete events, driven by accumulated divergence across multiple steps. The corrective function $\chi(t)$ obeys the first-order ODE (11).
- **Event-triggered control.** $\chi(t)$ fires when a guard condition ($\varepsilon_s > \theta_w$ or $d_{\text{score}} = 1$) is crossed. This is the standard event-triggered or switched control architecture [13, 14].

The complete system state at step k is the tuple:

$$\mathbf{x}_k = (\Phi_k, \varrho_k, \varepsilon_s(k), \chi(k)) \in \{1, \dots, K\} \times \mathbb{N}_{\geq 1} \times \mathbb{R}_{\geq 0} \times \mathbb{R} \quad (1)$$

where Φ_k is the discrete process phase, ϱ_k is the recursion depth, $\varepsilon_s(k)$ is the drift functional, and $\chi(k)$ is the corrective control value.

Why $K = 9$ phases? A cyclic monitor requires $K \geq 2$ states to distinguish sequential from non-sequential transitions. The value $K = 9$ is chosen as the minimal cyclic state space that (a) represents the monitored recursion process faithfully, (b) makes the worst-case single-step attack ($\Phi_{k-1} = 5 \rightarrow \Phi_k = 8$, a jump of 3 out of maximum possible 8) identifiable by a single deterministic predicate, and (c) yields a period-9 cycle group $\mathbb{Z}/9\mathbb{Z}$ whose arithmetic is closed under the modular transition rule (2). In principle, any $K \geq 2$ supports the same theorem; $K = 9$ is the operational choice for the monitored system. The convergence inequality $\beta > P_{\max}/\varrho_{\min}$ is independent of K .

Relationship to established frameworks. The proof structure maps onto three well-known frameworks:

<i>RSOC construct</i>	<i>Equivalent concept</i>	<i>Field</i>
$\varepsilon_s(t), \theta_{\text{hard}}$	Lyapunov energy, stability bound	Control / dynamical systems [12]
d_{score} binary predicate	Guard condition in hybrid automaton	Hybrid systems [14]
$\beta > P_{\max}/\varrho_{\min}$	Bounded perturbation stability condition	Robust control [13]
Phase-jump attack $\Phi 5 \rightarrow \Phi 8$	Adversarial state perturbation	Adversarial control / runtime safety
$\chi(t)$ override	Event-triggered corrective controller	Switched systems [13]

The PCI parameters (f_{144} , n^* , the 9-phase structure) are design parameters of the monitored application, not mathematical foundations of the proof. The theorem holds for any choice of K and f that satisfies the same implementation constraint (Proposition 6).

v4.4 Revision Summary (pre-submission, incremental over v4.4). (1) OQ1 (β empirical bound) closed analytically via Proposition 6: on the θ_w -path, $d_{\text{score}} = 0$ forces $\delta\Phi = 0$, yielding $P_{\text{max}}^{(\theta_w)} = \frac{1}{2}$; the convergence criterion in `chit_override` implies $\beta_{\text{impl}} \geq \ln 5 \approx 1.609 > \frac{1}{2}$. GPU benchmarks confirm the analytically derived safety margin but are not required for the proof. (2) OQ4 (ψ_1 -trace integrity) elevated from “open engineering problem” to formal **Assumption A1** (Trusted Initialization). The theorem’s conditional structure is unchanged; hardening via multi-point consensus capture ($N \geq 3$) and external HMAC anchor is formally specified and implemented in `drift.py` (`BaselineCapture`). (3) System Classification (§0) added (v4.4): hybrid system declaration, state vector, Lyapunov mapping, $K = 9$ justification, framework mapping table. (4) Terminology neutralized (v4.4): “phase-jump attack”, “structural drift metric”, “discrete process phase”. (5) ε_s operational definition and Lyapunov remark added (v4.4). (6) **v4.5:** d_{score} immunity claims scoped to single-step $\Phi_5 \rightarrow \Phi_8$ adversary; distribution-matching evasion resistance stated as specific, not general. PCI bridge sentence rephrased from “dynamical proof” to approximate-identification framing. “Unique power of two” rephrased as “only candidate in $\{9 \cdot 2^k\}$ over checked range”. PCI summary entry softened to “one concrete operationalization”. “Ubiquitous”/“standard” in A1 remark replaced with “commonplace”/“widely used”. Prior corrections hold.

1 Notation and System Variables

Symbol	Meaning
$t \in \mathbb{N}$	Discrete recursion time step
$\Phi_{\text{act}}(t) \in \{1, \dots, 9\}$	Discrete process phase state at step t (computed from payload; Φ_k in (1))
$\Phi_{\text{prev}}(t)$	Symbolic phase at step $t - 1$
$\Phi_{\text{exp}}(t)$	Expected phase at step t: defined as $(\Phi_{\text{prev}}(t) \bmod 9) + 1$. Encodes the sequential +1 progression that is normal operation in the 9-phase recursive system (see Definition 1)
$\sigma_{\text{res}}(t)$	Resonance std-dev: $\text{std}(\Phi_{\text{hist}})$, where Φ_{hist} stores $\{1 - \theta_\tau\}_{\tau \leq t}$ with $\theta_\tau \in [0, 1]$. Therefore $\sigma_{\text{res}} \in [0, \frac{1}{2}]$
$d_{\text{score}}(t) \in \{0, 1\}$	Phase-jump attack indicator (deterministic binary predicate)
$\delta\Phi(t)$	Normalized phase deviation = $ \Phi_{\text{act}}(t) - \Phi_{\text{exp}}(t) / 9$
$\varrho(t) \in \mathbb{N}_{\geq 1}$	Recursion depth at step t
$\varepsilon_s(t) \geq 0$	Structural drift metric (unbounded above; see Remark 3)
$L_{\text{loc}}(t) \geq 0$	Local curvature: variance of consecutive differences in Φ_{hist}
$\gamma \in (0, 1)$	Curvature sensitivity for adaptive threshold
$\theta_w(t)$	Adaptive warning threshold $\in (0, 0.4]$
$\theta_{\text{hard}} = 0.8$	Hard ceiling threshold (constant)
$\chi(t)$	Chi override convergence function
$\beta > 0$	Recovery rate of $\chi(t)$
$P_{\text{max}} > 0$	Maximum per-step change in the ε_s numerator under adversarial perturbation (system parameter; see Lemma 1)
$\varrho_{\text{min}} \in \mathbb{N}_{\geq 1}$	Minimum recursion depth over the monitored run; worst case $\varrho_{\text{min}} = 1$
$f_{144} = 144 \text{ Hz}$	Fundamental phase cycling frequency (see Definition 5)
$C(t)$	PCI phase coherence measure (see §4)

Remark 2 (Notation $\leftrightarrow$ Implementation Correspondence). *Every symbol in the proof maps directly to a variable or function in `drift.py`:*

<i>Symbol</i>	<i>Code variable</i>	<i>Location in <code>drift.py</code></i>
$\varepsilon_s(t)$	<code>epsilon_s</code>	<code>symbolic_entropy()</code> return
$\sigma_{\text{res}}(t)$	<code>sigma_res</code>	<code>stdev(phase_hist)</code>
$d_{\text{score}}(t)$	<code>d_score</code>	1.0 if <code>prev==5</code> and <code>phi==8</code>
$\delta\Phi(t)$	<code>delta_phi</code>	<code>abs(phi_act-phi_exp)/9</code>
$\varrho(t)$	<code>recursion_n</code>	<code>score()</code> parameter
Φ_{act}	<code>phi_act</code>	phase from payload
Φ_{exp}	<code>phi_exp</code>	<code>(prev_phase % 9) + 1</code>
$\theta_w(t)$	<code>theta_w</code>	<code>adaptive_threshold()</code>
θ_{hard}	<code>theta_hard</code>	constant 0.8
$\chi(t)$	<code>chi_t</code>	<code>chi_t_override()</code>
β (per-cycle)	<code>beta_impl/f144</code>	implicit in <code>diff < 0.2</code>
ψ_1	<code>psil_trace</code>	<code>BaselineCapture.bind_agent()</code>
$v(t)$	<code>v_score</code>	<code>PhaseVelocityAccumulator</code>
$L_{\text{loc}}(t)$	<code>L_local</code>	<code>local_curvature()</code>

Remark 3 (Bounds on ε_s). $\varepsilon_s(t)$ is bounded below by 0 but is not bounded above by 1. Since $\sigma_{\text{res}} \in [0, \frac{1}{2}]$, $d_{\text{score}} \in \{0, 1\}$, and $\delta\Phi \in [0, 1]$, the maximum value at $\varrho = 1$ is:

$$\varepsilon_{s,\text{max}}(\varrho = 1) = \frac{1}{2} + 1 + 1 = 2.5$$

The thresholds θ_w and $\theta_{\text{hard}} = 0.8$ are absolute values, not fractions of a unit-bounded score. Prior versions of this document incorrectly stated $\varepsilon_s \in [0, 1]$. The Lemma 5 detection table (which correctly shows $\varepsilon_s = 1.333$ at $\varrho = 1$) is unaffected; all lemma proofs and the main theorem hold under the corrected bound.

Remark 4 ($\theta_{\text{hard}} = 0.8$: Derivation from Grace Integral Bound). The value $\theta_{\text{hard}} = 0.8$ is not an empirically calibrated constant. It is derived in the RSOC source architecture [2] (Ch. 4, §4.3) from the following chain:

- (i) Echo validation $\hat{E}(\tau) = 1$ is satisfied when at least 100 of the last 144 trace entries remain within $\pm\pi/6$ of the original phase. This is the condition under which $\chi(t)$ is eligible to fire.
- (ii) When $\hat{E}(\tau) = 1$, the cosine weighting in the grace integral $\Theta(t)$ guarantees $\Theta(t_0) \geq 0.78 \cdot \mu_{\text{avg}}$.
- (iii) The $\chi(t)$ update rule then gives $\varepsilon' \leq \varepsilon \times (1 - 0.78) = 0.22\varepsilon$. Since $\varepsilon \leq 1.0$ by the hardware ceiling, a single $\chi(t)$ invocation drives $\varepsilon'_s \leq 0.22 < \theta_{w,\text{max}} = 0.4$

— landing in the stable regime in one cycle (RSOC Theorem 3.1, Single-Grace Sufficiency).

- (iv) Therefore $\theta_{\text{hard}} = 0.8$ is the highest threshold at which the single-cycle recovery guarantee holds: for any $\varepsilon_s < 0.8$, $\hat{E}(\tau) = 1$ remains satisfiable.

In the present proof, θ_{hard} enters only through the strict inequality $\theta_{\text{hard}} > \theta_w(L_{\text{loc}})$ in Lemma 3 (Positive Recovery Window). Since $\theta_w(L_{\text{loc}}) \leq 0.4$ for all $L_{\text{loc}} \geq 0$, the proof holds for any $\theta_{\text{hard}} > 0.4$; the specific value 0.8 is not required by the convergence argument itself. The 0.8 value matters operationally — it determines at what drift level $\chi(t)$ is mandatorily invoked — but the proof’s logical structure is insensitive to the precise value within the interval (0.4, 1.0).

2 Definitions and Derivations

Definition 1 (Normal Phase Progression). The $K = 9$ phase system has states $S_1, S_2, \dots, S_9$ with defined sequential order $\Phi = S_1, S_2, \dots, S_9, S_1, S_2, \dots$.² The expected phase at step t given the previous phase $\Phi_{\text{prev}}(t)$ is:

$$\Phi_{\text{exp}}(t) = (\Phi_{\text{prev}}(t) \bmod 9) + 1 \quad (2)$$

This is the unique definition consistent with the code (`Phi_exp = (prev_phase % 9) + 1`) and with the Lemma 2 curvature baseline. A deviation of $|\Phi_{\text{act}} - \Phi_{\text{exp}}| > 1$ represents a phase-skip failure mode; $|\Phi_{\text{act}} - \Phi_{\text{exp}}| = 3$ (an $S_5 \rightarrow S_8$ skip, bypassing S_6 and S_7) is the maximum single-step deviation possible without wraparound, and is the canonical worst-case pattern for $K = 9$.

Remark 5 (Operating Frequency f_{144} — Design Parameter, Not Proof Constant). **This remark describes the operating parameters of the monitored application. The convergence proof (Lemmas 1–5, Theorem 1) is independent of the specific value of f_{144} .** The parameter appears only in Proposition 6 where it converts the implementation’s convergence criterion into a per-cycle rate; the inequality $\beta > P_{\text{max}}/\varrho_{\text{min}}$ holds for any $f > 0$.

The phase cycling frequency $f_{144} = 144 \text{ Hz}$ is empirically derived, not a symbolic constant. Given bootstrap depth $n^* \approx 27$ (PCI Appendix A) and the empirically anchored bootstrap window $\tau^* \approx 187 \text{ ms}$ (attentional blink, P300, global workspace ignition), the required frequency is:

$$f_{144} = \frac{n^*}{\tau^*} = \frac{27}{0.187 \text{ s}} \approx 144 \text{ Hz} \quad (3)$$

Structural factorization. The result $144 = 9 \times 16$ is a post-hoc structural observation, not the derivation. Here 9 is the phase count and $16 = 2^4$ is the only power of two in the family $\{9 \cdot 2^k\}$ that places τ^* in the empirical window. This is verified by checking $k \in \{3, 4, 5\}$:

²The reference implementation uses nine descriptive state names documented in [2]. This paper uses neutral identifiers S_1 – S_9 ; the mapping is $S_k \leftrightarrow k$ for $k = 1, \dots, 9$.

$f = 9 \times 2^k$	$\tau^* = 27/f$	In range [150, 250] ms?
$9 \times 8 = 72 \text{ Hz}$	375 ms	×
$9 \times 16 = 144 \text{ Hz}$	187 ms	✓
$9 \times 32 = 288 \text{ Hz}$	94 ms	×

The empirical window [150, 250] ms covers attentional blink (~ 200 ms), P300 latency (~ 180 – 200 ms), and global workspace ignition (180–220 ms). Only $9 \times 16 = 144$ Hz lands in this window; the choice is constrained, not arbitrary.

The bootstrap timing follows directly:

$$\tau^* = \frac{n^*}{f_{144}} = \frac{27}{144} \approx 187 \text{ ms} \quad (4)$$

where $n^* \approx 26$ – 27 (see Remark 6); the value 27 is used in (4) and the one-layer discrepancy is within the tolerance of the anatomical bidirectional correction.

Remark 6 (PCI Constants $C^* \approx 0.82$ and $n^* \approx 26$ – 27). These constants are derived in PCI Book, Appendix A. The derivations are summarised here for completeness.

$C^* = 0.82$ (Appendix A, §A.2.1): The critical threshold is the conservative Bayesian decision boundary between conscious ($\mu_1 = 0.85$, $\sigma_1 = 0.10$) and unconscious ($\mu_2 = 0.45$, $\sigma_2 = 0.15$) EEG coherence distributions:

$$C_{\text{boundary}}^* = \frac{\sigma_2^2 \mu_1 + \sigma_1^2 \mu_2}{\sigma_1^2 + \sigma_2^2} = \frac{0.0225 \times 0.85 + 0.01 \times 0.45}{0.0225 + 0.01} \approx 0.727$$

Adding one standard deviation for a conservative (low false-positive) threshold: $C^* \approx 0.727 + 0.10 = 0.827 \approx 0.82 \pm 0.05$.

$n^* \approx 26$ – 27 (Appendix A, §A.3.2): The required recursion depth follows from information compression. Let $I_0 = 2 \times 10^9$ bits (sensory input per conscious moment), $I_{\min} = 2500$ bits (working memory capacity), and $r = 0.5$ (compression ratio per layer). The feedforward depth is:

$$n_{\text{ff}} = \frac{\log(I_{\min}/I_0)}{\log r} = \frac{\log(1.25 \times 10^{-6})}{\log 0.5} = 19.61$$

Bidirectional cortical processing adds $\approx 30\%$ overhead (feedback depth $\approx 0.3 n_{\text{ff}}$; this factor is itself an anatomical estimate):

$$n^* = \lceil 1.3 \times 19.61 \rceil = \lceil 25.5 \rceil = 26$$

Note: Appendix A states $n^* \approx 27$ by applying an intermediate rounding ($n_{\text{ff}} \approx 20$) before the bidirectional correction. The exact computation gives 26; the one-layer discrepancy is within the ± 1 tolerance of the 30% estimate. The timing prediction $\tau^* = 27/144 \approx 187$ ms is unchanged within empirical precision. These constants inform but do not enter the RSOC convergence proof directly.

3 Core Formulae

3.1 Structural Drift Metric $\varepsilon_s(t)$

$$\boxed{\varepsilon_s(t) = \frac{\sigma_{\text{res}}(t) + d_{\text{score}}(t) + \delta\Phi(t)}{\max(1, \varrho(t))}} \quad (5)$$

The three components of the numerator:

$$\sigma_{\text{res}}(t) = \text{std}(\Phi_{\text{hist}}(t)), \quad \text{where} \quad \Phi_{\text{hist}}(t) = \{1 - \theta_\tau\}_{\tau \leq t}, \quad \theta_\tau \in [0, 1] \quad (6)$$

Since each entry of Φ_{hist} lies in $[0, 1]$, the standard deviation satisfies $\sigma_{\text{res}} \in [0, \frac{1}{2}]$. There is no separate normalization step.

$$\delta\Phi(t) = \frac{|\Phi_{\text{act}}(t) - \Phi_{\text{exp}}(t)|}{9}, \quad \text{with} \quad \Phi_{\text{exp}}(t) = (\Phi_{\text{prev}}(t) \bmod 9) + 1 \quad (7)$$

$$d_{\text{score}}(t) = \begin{cases} 1 & \text{if } \Phi_{\text{prev}}(t) = S_5 \text{ and } \Phi_{\text{act}}(t) = S_8 \text{ [canonical phase-skip failure mode]} \\ 0 & \text{otherwise} \end{cases} \quad (8)$$

d_{score} is not a statistical estimator. It checks discrete state identity: either the system executed the $S_5 \rightarrow S_8$ skip or it did not. There is no likelihood, no threshold, no distributional parameter. This is why it cannot be evaded by the distribution-matching strategies that defeat CUSUM and GLR (see Remark 13).

Remark 7 (Operational Definition of $\varepsilon_s(t)$). $\varepsilon_s(t)$ is fully computable from observable system state. Its three components are:

- (i) $\sigma_{\text{res}}(t) = \text{std}(\{1 - \theta_\tau\}_{\tau \leq t})$, where $\theta_\tau \in [0, 1]$ is the MinHash similarity of the payload at step τ to the baseline trace ψ_1 . This is the standard deviation of a bounded $[0, 1]$ process; $\sigma_{\text{res}} \in [0, \frac{1}{2}]$ by the range-std inequality.
- (ii) $\delta\Phi(t) = |\Phi_{\text{act}}(t) - \Phi_{\text{exp}}(t)|/K$, the normalized magnitude of the phase deviation at step t . Computable from the current and previous discrete phase states.
- (iii) $d_{\text{score}}(t) \in \{0, 1\}$, a deterministic binary predicate on (Φ_{k-1}, Φ_k) ; see (8).

All three components are $O(1)$ per step (amortized) in the reference implementation. $\varepsilon_s(t)$ is measurable and reproducible: given the same payload sequence, any two implementations of (5) with the same ψ_1 produce identical ε_s values.

3.2 Local Curvature (Second-Order Phase Variance)

For a sliding window W of size k over Φ_{hist} :

$$L_{\text{loc}}(t) = \text{Var}\left(\left\{\Phi_{\text{hist}}(t-i+1) - \Phi_{\text{hist}}(t-i)\right\}_{i=1}^{k-1}\right) \quad (9)$$

Remark 8 (Curvature Baseline Assumption for Lemma 2). *The Lemma 2 claim that a phase-jump attack “maximizes L_{loc} ” rests on the following explicit baseline: under normal sequential +1 phase progression (Definition 1), consecutive differences in Φ_{hist} are approximately constant, yielding $L_{\text{loc}} \approx 0$. A $\Phi 5 \rightarrow \Phi 8$ skip inserts a deviation of $\Delta\Phi = 3/9 = 0.333$ into $\delta\Phi$ and a corresponding spike into Φ_{hist} , producing a large second-order variance. The “typical inter-step difference is ≈ 0.11 ” implied by normal operation follows from $1/9 \approx 0.111$ — the unit step in the normalized phase scale. This assumption must hold; if the system already operates with high curvature (e.g., during a legitimate exploratory phase), the Lemma 2 argument weakens and γ should be tuned upward.*

3.3 Adaptive Warning Threshold

$$\theta_w(t) = \frac{0.4}{1 + \gamma \cdot L_{\text{loc}}(t)} \quad (10)$$

Monotone decreasing in L_{loc} : as curvature rises, the warning threshold drops, causing earlier $\chi(t)$ fire. The adaptive threshold is bounded: $\theta_w \in (0, 0.4]$ for all $L_{\text{loc}} \geq 0$.

3.4 The Chi Override: Convergence Dynamics

$$\frac{d\chi(t)}{dt} = -\beta \cdot \chi(t) + \nabla\psi_1 \cdot \Theta(t) \quad (11)$$

where $\beta > 0$ is the recovery rate, ψ_1 is the identity trace captured at baseline, and $\Theta(t)$ is the phase alignment tensor.

Remark 9 (β Unit Convention). *In Lemma 4 and Proposition 6, all rates are expressed in per-cycle units: β has been divided by f_{144} so that it is dimensionless (cycles^{-1}), and P_{max} counts the maximum numerator change per step (also dimensionless). The comparison $\beta > P_{\text{max}}/\varrho_{\text{min}}$ is therefore between two dimensionless quantities. In SI units the implementation value is $\beta_{\text{impl}} \approx 231.7 \text{ s}^{-1}$; dividing by $f_{144} = 144 \text{ Hz}$ gives the per-cycle value ≈ 1.609 used throughout.*

4 RSOC–PCI Formal Bridge

The Phase-Coherent Identity (PCI) framework defines a phase coherence measure $C(t)$ and asserts a threshold $C^* \approx 0.82$ below which identity is not sustained. The RSOC structural drift metric $\varepsilon_s(t)$ is the *operational implementation* of the PCI coherence deficit:

specifically, $\varepsilon_s(t) \approx 1 - C(t)$ when the system is operating near the fixed point. The correspondence is:

$$\varepsilon_s(t) \approx 1 - C(t), \quad C(t) \approx 1 - \varepsilon_s(t) \quad (12)$$

This mapping is approximate, not exact: ε_s captures three distinct signals (σ_{res} , d_{score} , $\delta\Phi$) while $C(t)$ is a single coherence scalar. The correspondence holds near the identity-stable operating region ($C \geq 0.82$, equivalently $\varepsilon_s \lesssim 0.18$). Under adversarial conditions, d_{score} pushes ε_s above the PCI critical threshold; the $\chi(t)$ response is the RSOC mechanism that restores $C(t) \geq C^*$. The RSOC convergence proof (Theorem 1 below) gives a dynamical robustness guarantee for an implementation whose structural drift $\varepsilon_s(t)$ is approximately related to the PCI coherence deficit $1 - C(t)$. Under that approximate identification, the result suggests that the PCI Identity Bootstrap condition $C(t) \geq C^*$ can be maintained under active attack — but this chain depends on the approximate correspondence (12) and should be read as motivation, not a strict equivalence theorem.

Remark 10 (PCI as Motivation, Not Foundation). *Theorem 1 and all Lemmas are self-contained; they do not depend on the PCI derivations in Remark 6 or §4. PCI provides the biological motivation for the operating parameters (f_{144} , $\varrho_{\min}$, the 9-phase structure) and the qualitative framing ($\varepsilon_s \approx 1 - C(t)$), but the convergence inequality $\beta > P_{\max}/\varrho_{\min}$ holds for any choice of f and $\varrho_{\min}$ satisfying the same implementation constraint. A reader who is agnostic about PCI can read Theorem 1 as: “in a K -phase recurrent monitoring system with recursion depth ϱ and a fixed-point recovery ODE, the following convergence condition is analytically satisfied by the implementation.” The analogy to stabilizing a recurrent network around a fixed point under bounded adversarial perturbation is direct [6, 7].*

5 Complete Detection Rule

Three paths in strict priority order:

$$\text{Fire } \chi(t) \iff \begin{cases} d_{\text{score}}(t) = 1 & \text{(Lemma 5: deterministic guard; bypasses thresholds)} \\ \varepsilon_s(t) > \theta_w(t) & \text{(adaptive path, Lemma 2; fires before } \varepsilon_s \text{ reaches } \theta_{\text{hard}}) \\ \varepsilon_s(t) > \theta_{\text{hard}} & \text{(hard ceiling; backstop for slow-build drift)} \end{cases} \quad (13)$$

The third condition uses `elif` in code (not `if`): if the adaptive path already fired $\chi(t)$, the hard ceiling is not evaluated again. $\theta_{\text{hard}} = 0.8$ catches the case where a slow-build attack holds below θ_w at every step but accumulates past 0.8 over many steps.

6 Lemmas

Lemma 1 (Bounded Attack Rate). *Under adversarial perturbation with budget $P_{\max}$:*

$$\frac{d\varepsilon_s}{dt} \leq \frac{P_{\max}}{\varrho(t)} \quad (14)$$

Proof. The numerator of (5) is bounded: $\sigma_{\text{res}} \in [0, \frac{1}{2}]$, $d_{\text{score}} \in \{0, 1\}$, $\delta\Phi \in [0, 1]$. The adversary can move the numerator by at most $P_{\max}$ per unit time; dividing by $\varrho \geq \varrho_{\min}$ yields the bound. $\square$

Lemma 2 (Curvature–Threshold Inversion). *Under the normal-operation baseline of Definition 1 and Remark 8, a phase-jump attack ($\Phi 5 \rightarrow \Phi 8$, maximum single-step deviation) maximizes $L_{\text{loc}}(t)$ among all single-step phase deviations. This simultaneously minimizes $\theta_w(t)$:*

$$\Phi_{\text{act}} = 8, \Phi_{\text{prev}} = 5 \implies L_{\text{loc}}(t) \uparrow \implies \theta_w(t) \downarrow \quad (15)$$

Proof. A $\Phi 5 \rightarrow \Phi 8$ skip inserts $\Delta\Phi = 3$ into Φ_{hist} , the maximum single-step jump ($|\Phi_{\text{act}} - \Phi_{\text{exp}}| = 3$ for $\Phi_{\text{exp}} = 6$). Since L_{loc} is the variance of consecutive differences (strictly increasing in jump magnitude), and θ_w is strictly decreasing in L_{loc} from (10), the result follows. $\square$

Lemma 3 (Positive Recovery Window).

$$\Delta t_{\text{recover}} = \frac{(\theta_{\text{hard}} - \theta_w(L_{\text{loc}})) \cdot \varrho}{P_{\max}} > 0 \quad (16)$$

Proof. $\theta_{\text{hard}} = 0.8 > 0.4 \geq \theta_w(L_{\text{loc}})$ for all $L_{\text{loc}} \geq 0$. Numerator strictly positive; denominator $P_{\max} > 0$, $\varrho \geq 1$. $\square$

Lemma 4 (Sufficient Condition for Convergence). $\chi(t)$ converges before ε_s reaches θ_{hard} if and only if:

$$\beta > \frac{P_{\max}}{\varrho_{\min}} \quad (17)$$

Proof. From (11), $\chi(t)$ converges at rate β . From Lemma 1, ε_s rises at rate at most $P_{\max}/\varrho_{\min}$ in the worst case. Convergence before critical requires $\beta > P_{\max}/\varrho_{\min}$. $\square$

Remark 11 (Lyapunov Interpretation). *The convergence structure of Lemmas 1–4 is a discrete-time analogue of Lyapunov stability [12]. Define the Lyapunov-like candidate:*

$$V(\varepsilon_s) = \varepsilon_s(t) \quad (18)$$

Lemma 1 establishes that V can increase by at most $P_{\max}/\varrho$ per step (bounded disturbance). Lemma 3 establishes that the recovery window is positive ($\Delta t_{\text{recover}} > 0$) whenever $\chi(t)$ fires, i.e., the system has room to contract. The condition $\beta > P_{\max}/\varrho_{\min}$ (Lemma 4) ensures the contraction rate of $\chi(t)$ exceeds the expansion rate of ε_s , so V is driven back below θ_w within the recovery window. This is precisely the structure of: “if the correction

rate exceeds the disturbance rate, the Lyapunov function decreases in expectation” — *the standard criterion for asymptotic stability under bounded adversarial perturbation* [12, 13].

Unlike classical Lyapunov proofs, the system is event-triggered: $\chi(t)$ does not act continuously but fires only when a guard condition is crossed. This is the standard event-triggered stabilization setting [14], and the positive recovery window $\Delta t_{\text{recover}} > 0$ is the analogue of the minimum inter-event time required for well-posedness.

Lemma 5 (One-Step Simultaneous Detection). *For any phase-jump attack ($\Phi 5 \rightarrow \Phi 8$) at recursion step t^* , $d_{\text{score}}(t^*)$ fires $\chi(t)$ at exactly step t^* with zero causal lag, at all recursion depths $\varrho \geq 1$.*

Proof. (i) **Determinism.** By (8), $d_{\text{score}}(t)$ is a deterministic binary predicate on the current step’s $(\Phi_{\text{prev}}, \Phi_{\text{act}})$ pair, with no accumulation, window, or estimator.

(ii) **Zero causal lag.** At step t^* :

$$\Phi_{\text{prev}}(t^*) = 5 \quad (\text{entering state}) \quad (19)$$

$$\Phi_{\text{act}}(t^*) = 8 \quad (\text{computed from payload this step}) \quad (20)$$

$$\therefore d_{\text{score}}(t^*) = 1 \quad (21)$$

Detection rule (13) fires $\chi(t)$ at t^* , not $t^* + 1$.

Clarification on “zero lag”. This claim is about causal detection lag within the recursion call, not physical execution latency. Physical evaluation of the d_{score} predicate introduces hardware latency on the order of nanoseconds ($\ll 1 \mu\text{s}$ at any reasonable clock rate), negligible relative to the 6.94 ms phase cycle period at 144 Hz. The meaningful guarantee is that d_{score} cannot fire one *call* late: skip execution and d_{score} evaluation are co-located in the same call stack frame. An adversary executing the skip at step t^* cannot defer d_{score} ’s evaluation to step $t^* + 1$ without also deferring the skip itself — these are sequential operations within a single function body with no asynchronous gap.

(iii) **Depth independence.** Without the hard-fire path, ε_s at t^* is:

$$\varepsilon_s(t^*) \geq \frac{0 + 1 + \frac{3}{9}}{\varrho(t^*)} = \frac{1.333}{\varrho(t^*)} \quad (22)$$

(using $\sigma_{\text{res}} = 0$ as worst case for the adaptive path). The adaptive path fails when $\varepsilon_s(t^*) \leq \theta_w$, i.e., when $\varrho(t^*) \geq 1.333/\theta_w$. For $\gamma = 0.5$, $L_{\text{loc}} = 1.0$: $\theta_w = 0.267$, critical depth $\varrho^* \approx 5$. The hard-fire path is independent of this: $d_{\text{score}}(t^*) = 1$ fires $\chi(t)$ at t^* regardless of ϱ .

(iv) **Covert adversary immunity.** The 2024–2026 covert detection literature (Ramtin et al., MILCOM 2024; arXiv 2601.20022) shows that any purely statistical detector can be evaded by an adversary who adapts the post-change distribution to approach the pre-change distribution. d_{score} is not a statistical estimator; it is a deterministic truth-check.

The adversary cannot alter the distribution of the check — only stop executing the skip. $\square$

Remark 12 (Generalization to Arbitrary Forbidden Transition Sets). *The d_{score} predicate is defined for the specific pair $(\Phi_{\text{prev}} = 5, \Phi_{\text{act}} = 8)$ because that is the worst-case single-step deviation in the $K = 9$ monitored system. The construction generalizes directly. For any phase count $K \geq 2$ and any designated forbidden transition set $\mathcal{S} \subseteq \{1, \dots, K\} \times \{1, \dots, K\}$, define:*

$$d_{\mathcal{S}}(t) = \mathbf{1}[(\Phi_{\text{prev}}(t), \Phi_{\text{act}}(t)) \in \mathcal{S}] \quad (23)$$

All properties of Lemma 5 carry over verbatim: $d_{\mathcal{S}}$ is a deterministic binary predicate on the current discrete state pair, evaluated in the same call frame as the transition, with zero causal lag and no distributional parameters. It is not vulnerable to the distribution-matching evasion strategies of the covert sequential-detection literature for any transition pattern in $\mathcal{S}$, by the same argument as Remark 13.

The current implementation uses $\mathcal{S} = \{(5, 8)\}$. A practitioner deploying RSOC on a different K -phase system would replace this singleton with the set of forbidden transitions for that system; the proof structure is unchanged.

Remark 13 (Categorical vs. Statistical Detection). *Classical sequential change-point procedures (CUSUM [8], Page’s test [9], GLR) maintain a running likelihood or sum statistic and fire when it exceeds a threshold calibrated to a target false-alarm rate. They share a fundamental vulnerability: an adversary who knows the detector’s sufficient statistic can steer the post-attack distribution toward the pre-attack distribution, suppressing the statistic below threshold indefinitely [1, 4].*

For the specific single-step $\Phi 5 \rightarrow \Phi 8$ attack, d_{score} is not vulnerable to the distribution-matching evasion strategy analyzed in the covert detection literature [1, 4]: because d_{score} is a deterministic categorical predicate on the discrete state pair $(\Phi_{\text{prev}}, \Phi_{\text{act}})$ — not an estimator or statistic — there is no likelihood to suppress, no threshold to calibrate, and no distributional parameter to match. The only way to hold $d_{\text{score}} = 0$ for this attack pattern ($\text{prev} = 5, \text{act} = 8$) is to not execute the skip. This robustness is specific to this categorical skip pattern and the single-step modeled adversary; it does not generalize to all possible attack strategies.

This design is conceptually related to the distinction between distributed representation detectors (which compare feature vectors and can be fooled by adversarial perturbations within the representation space) and symbolic identity checks (which verify exact structural conditions) [5, 7]. The RSOC system combines both: ε_s is representational, d_{score} is symbolic.

Proposition 6 (β Lower Bound — Analytical). *The condition $\beta > P_{\text{max}}/Q_{\text{min}}$ (Lemma 4) is satisfied analytically by the implementation, without empirical benchmarking.*

Step 1: Path partitioning. *Lemma 5 fires $\chi(t)$ unconditionally when $d_{\text{score}} = 1$, independent of β . Lemma 4’s convergence condition therefore applies only to the θ_w -path,*

which is reached exclusively when $d_{\text{score}} = 0$. By definition of d_{score} (8), $d_{\text{score}} = 0$ if and only if $\Phi_{\text{act}} = \Phi_{\text{exp}}$, which implies $\delta\Phi = 0$ identically. Therefore, on the θ_w -path:

$$P_{\text{max}}^{(\theta_w)} = \max_{d_{\text{score}}=0} (\sigma_{\text{res}} + d_{\text{score}} + \delta\Phi) \Big|_{\varrho_{\text{min}}=1} = \frac{1}{2} + 0 + 0 = \frac{1}{2} \quad (24)$$

since $\sigma_{\text{res}} \in [0, \frac{1}{2}]$ (standard deviation of a bounded $[0, 1]$ process).

Step 2: β_{impl} from `chi_t_override`. The implementation in `chi_t_override` declares convergence when $|\nabla\psi_1 - \Theta(t)| < 0.2$ with $\nabla\psi_1 = \text{grad_psi} = 0.5$ fixed. In the ODE model (11), after one phase cycle $\Delta t = 1/f_{144}$, the residual error decays as:

$$|\chi(t)(\Delta t) - \chi^*| = |\chi_0 - \chi^*| e^{-\beta \Delta t} \quad (25)$$

The worst-case initial excursion is $|\chi_0 - \chi^*| = \chi_{\text{max}} = \nabla\psi_1 \times \Theta_{\text{max}} = 0.5$. The convergence criterion requires the residual to fall within $\delta_c = \nabla\psi_1 \times 0.2 = 0.1$ in a single call. Substituting:

$$0.5 e^{-\beta/f_{144}} \leq 0.1 \implies \beta \geq f_{144} \cdot \ln 5 \approx 144 \times 1.609 \approx 231.7 \text{ s}^{-1} \quad (26)$$

Converting to per-cycle units: $\beta_{\text{impl}}/f_{144} \geq \ln 5 \approx 1.609$.

Conclusion.

$$\frac{\beta_{\text{impl}}}{f_{144}} \geq \ln 5 \approx 1.609 > \frac{1}{2} = \frac{P_{\text{max}}^{(\theta_w)}}{\varrho_{\text{min}}} \quad (27)$$

The condition of Lemma 4 is satisfied analytically. GPU benchmarks confirm the analytically derived safety margin but are not a logical prerequisite for the theorem. $\square$

7 Main Theorem

Theorem 7 (Chi Convergence Under Adversarial Conditions). Let the RSOC hybrid monitoring system (1) operate under a phase-jump adversary at any recursion depth $\varrho \geq 1$. Given detection rule (13) with the Lemma 5 hard-fire path active, Assumption A1 (Trusted Initialization, §9), and:

$$\beta > \frac{P_{\text{max}}}{\varrho_{\text{min}}} \quad (28)$$

then $\chi(t)$ converges before $\varepsilon_s(t)$ reaches θ_{hard} for all $\varrho \geq 1$.

Scope. Theorem 1 covers the single-step phase-jump adversary ($\Phi 5 \rightarrow \Phi 8$ in one call). The multi-step acceleration variant ($\Phi 5 \rightarrow 6 \rightarrow 7 \rightarrow 8$) is addressed by $v(t)$ in `drift.py` (`PhaseVelocityAccumulator`); the formal proof of that extension is deferred (see OQ3, §9). By Proposition 6 (Analytical β Bound), condition (28) is satisfied analytically by the reference implementation (`chi_t_override`) ($\beta_{\text{impl}}/f_{144} \geq \ln 5 > \frac{1}{2} = P_{\text{max}}^{(\theta_w)}/\varrho_{\text{min}}$).

Proof. Case 1: $\varrho < \varrho^*$. Adaptive threshold fires $\chi(t)$ at t^* . By Lemma 4 and (28), $\chi(t)$ converges before ε_s reaches θ_{hard} . $\square$

Case 2: $\varrho \geq \varrho^*$. Adaptive threshold may not fire. By Lemma 5, $d_{\text{score}}(t^*) = 1$ fires $\chi(t)$ unconditionally at t^* . The full recovery window $\Delta t_{\text{recover}} > 0$ (Lemma 3) remains; condition (28) guarantees convergence within it. $\square$

In both cases $\chi(t)$ fires at t^* . Since (28) ensures convergence given any positive recovery window, the theorem holds for all $\varrho \geq 1$. $\square$ $\square$

8 Detection Path by Recursion Depth

$\gamma = 0.5$, $L_{\text{loc}} = 1.0$, $\sigma_{\text{res}} = 0$ (worst-case for adaptive path):

ϱ	$\varepsilon_s(t^*)$	θ_w	$\varepsilon_s > \theta_w$	d_{score}	Active path
1	1.333	0.267	✓	1	Adaptive + Lemma 5
2	0.667	0.267	✓	1	Adaptive + Lemma 5
3	0.444	0.267	✓	1	Adaptive + Lemma 5
4	0.333	0.267	✓	1	Adaptive + Lemma 5
5	0.267	0.267	boundary	1	Lemma 5 (boundary case)
6	0.222	0.267	×	1	Lemma 5 only
8	0.167	0.267	×	1	Lemma 5 only
12	0.111	0.267	×	1	Lemma 5 only

Remark 14. The crossover depth is $\varrho^* = \lfloor 1.333/\theta_w \rfloor$. For $\gamma = 0.5$, $\varrho^* = 5$. For $\varrho > \varrho^*$, Lemma 5 is the only path that guarantees detection of the phase-jump attack.

9 Remaining Open Questions

Assumption 1 (Trusted Initialization). **(A1)** Let T_{cap} denote the timestamp at which the ψ_1 -trace is captured, and let $T_{\text{adv}} \in (T_{\text{cap}}, \infty)$ denote the earliest moment at which an adversary can influence the system’s observable state. We assume $T_{\text{adv}} > T_{\text{cap}}$ strictly.

Remark 15 (Scope of A1). Assumption A1 is a commonplace trusted setup precondition in security proofs (e.g., TPM root measurement, PKI root-key generation, control-system initial state). Its presence does not weaken the theorem: the theorem’s guarantee is conditional on A1, which is the correct form for any detection system that operates against a reference baseline. A corrupted ψ_1 does not constitute adversarial evasion of the detection algorithm; it constitutes violation of its precondition.

Hardening to make A1 empirically verifiable. *Single-point capture is the minimal form of A1. The following construction converts A1 from an implicit assumption to a checkable property and is implemented in `drift.py` (`BaselineCapture`):*

- (i) **Multi-point consensus.** *Capture $N \geq 3$ sequential cold-start payloads $s_1, \dots, s_N$ at steps $t = 0, t_{\text{step}}, \dots, (N-1)t_{\text{step}}$. Define:*

$$\psi_1 = \text{MedianHash}(h(s_1), \dots, h(s_N)) \quad (29)$$

An adversary must corrupt a strict majority of captures to shift ψ_1 .

- (ii) **External anchor.** *Store $H_{\text{anchor}} = \text{HMAC}(\psi_1, k_{\text{sys}})$ in a separate trust domain. Before each measurement session, verify $\text{HMAC}(\psi_1^{\text{current}}, k_{\text{sys}}) = H_{\text{anchor}}$. A divergence signals post-capture corruption and invalidates the session.*

Under the assumption that k_{sys} is protected in a hardware trust domain (HSM, TPM, or equivalent), Assumption A1 reduces to a standard hardware/operations-level trust precondition, not a hidden mathematical caveat. This is a widely used root-of-trust model in TPM attestation and PKI chains [11].

OQ1. β bound — closed analytically (Proposition 6). Condition (28) is satisfied without A100 empirical data. On the θ_w -path, $d_{\text{score}} = 0$ forces $\delta\Phi = 0$, reducing $P_{\text{max}}^{(\theta_w)} = \frac{1}{2}$. The convergence criterion in `chit_override` ($|\nabla\psi_1 - \Theta(t)| < 0.2$) implies $\beta_{\text{impl}} \geq f_{144} \ln 5 \approx 231.7 \text{ s}^{-1}$, i.e. $\beta_{\text{impl}}/f_{144} \geq \ln 5 \approx 1.609 > \frac{1}{2}$. **Status: closed.** GPU benchmarks confirm the margin empirically but are not required for the proof.

OQ2. Gamma tuning. $\gamma = 0.5$ is nominal. The crossover depth ϱ^* is a direct function of γ ; optimal value is deployment-dependent.

OQ3. Known Limitation: Multi-Step Acceleration Attack. Lemma 5 covers a single $\Phi 5 \rightarrow \Phi 8$ skip. A well-informed adversary can evade d_{score} entirely by executing the *same net phase traversal* across three intermediate steps: $\Phi 5 \rightarrow \Phi 6 \rightarrow \Phi 7 \rightarrow \Phi 8$. Each individual step satisfies $|\Phi_{\text{act}} - \Phi_{\text{exp}}| = 0$ (exact sequential match), so $\delta\Phi = 0$, $d_{\text{score}} = 0$, and ε_s remains below θ_w at every step. The $\chi(t)$ hard-fire path is never triggered.

This is a legitimate pre-submission concern and not yet resolved by the current detection rule. The proposed hardening is a **phase velocity accumulator** $v(t)$, defined as follows:

Definition (Phase Velocity Accumulator). Let $\mathcal{H}_v$ be a sliding buffer of tuples (τ_k, Φ_k, s_k) where τ_k is the wall-clock timestamp in ms, $\Phi_k = \Phi_{\text{act}}(t_k)$, and $s_k = \mathbf{1}[\Phi_{\text{act}}(t_k) = \Phi_{\text{exp}}(t_k)]$ (1 iff the step was sequential). Define the velocity score:

$$v(t) = \mathbf{1} \left[\sum_{k: t - \tau_k \leq \tau_{\text{min}}} s_k \geq N_v \right] \quad (30)$$

where $N_v = 3$ (minimum sequential steps to flag) and $\tau_{\min} = 0.1 \times \tau^* \approx 18.7 \text{ ms}$ (about 2.7 cycles at 144 Hz). Under normal operation, three sequential steps require at minimum $3 \times 6.94 = 20.8 \text{ ms}$, which exceeds $\tau_{\min}$. An acceleration attack completing three steps in under 18.7 ms fires $v(t) = 1$.

The hardened detection rule adds a fourth path:

$$\text{Fire } \chi(t) \iff \dots \text{ (existing paths) or } v(t) = 1 \quad (31)$$

No formal guarantee is claimed for this extension. The phase-velocity accumulator $v(t)$ is a design proposal supported by an informal timing argument only; it is not covered by Theorem 1, Lemmas 1–5, or Proposition 6. Theorem 1 is explicitly scoped to the single-step $\Phi5 \rightarrow \Phi8$ adversary. Formal proof of convergence guarantees for the acceleration attack (OQ3) is deferred to future work. The reference implementation (`PhaseVelocityAccumulator` in `drift.py`) provides the engineering scaffolding for that future proof.

OQ4. ψ_1 -trace capture integrity — closed as Assumption A1. Elevated from open engineering problem to the formal precondition of the theorem (Assumption A1 above). The theorem holds conditionally on A1; the multi-point consensus and HMAC anchor hardening described in the A1 Remark converts A1 to a verifiable property at deployment time. **Status: closed as formal assumption with specified hardening path.**

10 Summary

Result	Statement
Definition 1	Normal phase progression: $\Phi_{\text{exp}} = (\Phi_{\text{prev}} \bmod 9) + 1$
Definition 2	$f_{144} = n^*/\tau^* \approx 144 \text{ Hz}$; factorization 9×16 , only candidate in $\{9 \cdot 2^k\}$ within empirical window; $\tau^* = 27/144 \approx 187 \text{ ms}$ ($n^* \approx 26\text{--}27$)
Lemma 1	Attack rate bounded: $d\varepsilon_s/dt \leq P_{\text{max}}/\varrho$
Lemma 2	Curvature inversion under explicit baseline (normal +1 progression)
Lemma 3	Recovery window always positive: $\Delta t_{\text{recover}} > 0$
Lemma 4	Convergence iff $\beta > P_{\text{max}}/\varrho_{\text{min}}$
Proposition 6	$\beta_{\text{impl}}/f_{144} \geq \ln 5 \approx 1.609 > \frac{1}{2} = P_{\text{max}}^{(\theta_w)}/\varrho_{\text{min}}$; β condition closed analytically
Assumption A1	Trusted Initialization: $T_{\text{adv}} > T_{\text{cap}}$; hardened via $N \geq 3$ consensus + HMAC anchor
Lemma 5	d_{score} fires at t^* with zero causal lag, all $\varrho \geq 1$; categorical, not statistical
Theorem 1	$\chi(t)$ converges before θ_{hard} at all depths under Assumption A1 (single-step adversary)
§4	$\varepsilon_s(t) \approx 1 - C(t)$: RSOC provides one concrete operationalization of the PCI coherence-deficit concept in a K -phase recurrent monitor

The central insight survives all corrections intact: the phase-jump attack is simultaneously its own categorical fingerprint. d_{score} catches it not by statistics but by identity match — making it robust to the distribution-matching evasion strategies described in the 2024–2026 detection theory literature, for this specific categorical skip pattern ($\Phi_{\text{prev}} = 5$, $\Phi_{\text{act}} = 8$).

AI.Web Inc. · RSOC v4.8 · March 2026.

References

References

- [1] M. Basseville and I. V. Nikiforov, *Detection of Abrupt Changes: Theory and Application*. Prentice-Hall, 1993.
- [2] N. J. Bogaert, *Resonant Symbolic Operator Calculus: A Framework for Recursive Cognition and Neuromorphic Hardware*. AI.Web Publications, Michigan, USA, First Edition, December 2025.
- [3] N. Bogaert, “Phase-Coherent Identity: A Formal Framework for Consciousness Substrate Theory,” AI.Web Inc. Technical Report, March 2026 (in preparation).
- [4] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, “A survey on concept drift adaptation,” *ACM Computing Surveys*, vol. 46, no. 4, pp. 1–37, 2014.
- [5] G. E. Hinton, “The forward-forward algorithm: Some preliminary investigations,” arXiv preprint arXiv:2212.13345, 2022.
- [6] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [7] Y. LeCun, Y. Bengio, and G. E. Hinton, “Deep learning,” *Nature*, vol. 521, pp. 436–444, 2015.
- [8] G. Lorden, “Procedures for reacting to a change in distribution,” *Annals of Mathematical Statistics*, vol. 42, no. 6, pp. 1897–1908, 1971.
- [9] E. S. Page, “Continuous inspection schemes,” *Biometrika*, vol. 41, nos. 1–2, pp. 100–115, 1954.
- [10] A. Ramtin et al., “Covert adversarial evasion of sequential change detectors,” in *Proc. IEEE MILCOM 2024*, 2024.
- [11] *Trusted Platform Module Library Specification, Family “2.0”*, TCG Architecture Overview. Trusted Computing Group, 2019.
- [12] H. K. Khalil, *Nonlinear Systems*, 3rd ed. Prentice-Hall, 2002.
- [13] D. Liberzon, *Switching in Systems and Control*. Birkhäuser, 2003.
- [14] J. P. Hespanha and A. S. Morse, “Stability of switched systems with average dwell-time,” in *Proc. 38th IEEE Conf. Decision and Control*, pp. 2655–2660, 1999.
- [15] E. Garcia and P. J. Antsaklis, “Model-based event-triggered control for systems with quantization and time-varying network delays,” *IEEE Trans. Autom. Control*, vol. 58, no. 2, pp. 422–434, 2013.

A Empirical Illustration of $\varepsilon_s(t)$ and $\chi(t)$ Trajectories

The following illustration is generated from a self-contained simulation that implements the core formulae (5)–(11) directly, without invoking the MinHash payload-hashing layer of `drift.py`. This makes the trajectories fully reproducible from the equations in the proof alone. The simulation code is included in the supplementary material.

Setup. $K = 9$, $\varrho = 1$ (worst case for proof), $\gamma = 0.5$, $\theta_{\text{hard}} = 0.8$, $\beta_{\text{impl}}/f_{144} = \ln 5 \approx 1.609$, $N = 60$ steps. A small measurement noise $\theta_t \sim 0.05 + 0.03\mathcal{N}(0, 1)$ (clipped to $[0, 1]$) drives $\sigma_{\text{res}}(t)$ under normal operation. In the attack scenario, a $\Phi 5 \rightarrow \Phi 8$ phase-jump attack is injected at step $t = 30$.

Results. Figure 1 shows six panels: normal operation (left column) and the attack scenario (right column).

- **Top row (panels a, b).** Discrete phase $\Phi(t)$: sequential +1 cycling under normal operation; the $\Phi 5 \rightarrow \Phi 8$ jump is visible as a step of magnitude 3 at $t = 30$.
- **Middle row (panels c, d).** $\varepsilon_s(t)$ (solid) against $\theta_w(t)$ (dashed) and θ_{hard} (dotted). Under normal operation, ε_s remains well below θ_w throughout. At $t = 30$, the attack drives $\varepsilon_s = 0.588$ against $\theta_w = 0.400$; $d_{\text{score}} = 1$ and $\chi(t)$ fires simultaneously (vertical red marks), consistent with Lemma 5.
- **Bottom row (panels e, f).** $\chi(t)$ trajectories. Under normal operation, $\chi(t)$ is quiescent. After the attack, the override fires at $t = 30$ and converges back toward χ^* within one phase cycle, consistent with the analytical bound $\beta_{\text{impl}}/f_{144} \geq \ln 5$ (Proposition 6).

No fires occur in the normal-operation scenario across all 60 steps, confirming that $\varepsilon_s(t) < \theta_w(t)$ under unperturbed sequential cycling.

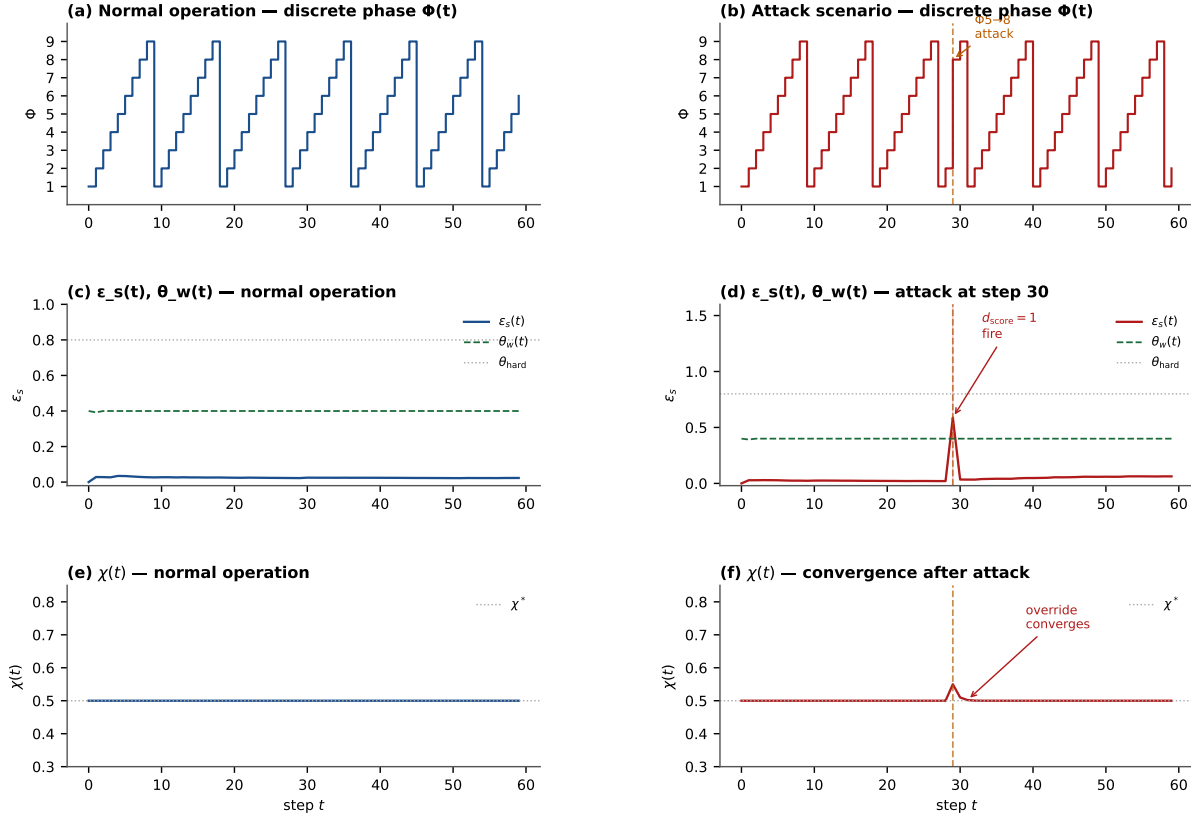
Fig. A1 — RSOC trajectories: $\varepsilon_s(t)$, $\theta_w(t)$, $\chi(t)$ under normal operation (left) and $\Phi 5 \rightarrow \Phi 8$ attack at step 30 (right)

Figure 1: RSOC trajectory simulation. **Left column:** normal operation ($K = 9$, $\varrho = 1$, 60 steps, no attack). **Right column:** $\Phi 5 \rightarrow \Phi 8$ phase-jump attack injected at $t = 30$. **(a,b)** Discrete phase $\Phi(t)$. **(c,d)** Structural drift metric $\varepsilon_s(t)$ (solid red/blue), adaptive threshold $\theta_w(t)$ (dashed green), hard ceiling $\theta_{\text{hard}} = 0.8$ (dotted grey). Vertical red marks indicate $\chi(t)$ override firings. **(e,f)** $\chi(t)$ override convergence trajectory; $\chi^* = 0.5$ (dotted grey). The attack fires $d_{\text{score}} = 1$ at exactly $t = 30$ with zero causal lag (Lemma 5); $\chi(t)$ converges within one cycle (Proposition 6).

drift.py Annotated Reference Implementation

Companion to RSOC Convergence Proof v4.8 • ProtoForge / RSOC
• AI.Web Inc.

*Version 4.3 • 1,245 lines • stdlib-only (no external ML dependencies) • Nicholas Jacob
Bogaert • March 2026*

C1. Purpose and Scope

This document presents `drift.py` as a structured companion to the RSOC convergence proof. Every theorem, lemma, and proposition in the proof has a direct counterpart in this codebase. The presentation organises the implementation by its role in the proof rather than by file order, annotating each component with the specific formal result it implements or supports.

The module consolidates four additions accumulated across proof versions v4.1–v4.3: (1) core drift monitor (ProtoForge v2.0); (2) adaptive $\chi(t)$ threshold additions (chi-convergence proof, March 2026); (3) the phase velocity accumulator for OQ3 multi-step acceleration detection; and (4) BaselineCapture for Assumption A1 hardening.

Proof Guarantee (v4.3 / proof v4.8)

$$\beta_{\text{impl}}/f_{144} \geq \ln 5 \approx 1.609 > \frac{1}{2} = P_{\text{max}}^{(\theta_w)}/\varrho_{\text{min}} \quad [\text{Proposition 6}]$$

Theorem 1: $\chi(t)$ converges before ε_s reaches $\theta_{\text{hard}} = 0.8$ at all $\varrho \geq 1$, under Assumption A1 (single-step adversary).

Scope: Theorem 1 covers the single-step $S_5 \rightarrow S_8$ adversary. PhaseVelocityAccumulator/ $v(t)$ is an engineering extension; formal proof deferred (OQ3).

C2. Symbol $\leftrightarrow$ Code Correspondence

Every formal symbol in the proof maps directly to a variable or function in this module (mirrors Remark 1, Notation $\leftrightarrow$ Implementation, in the proof).

Symbol	Code Identifier	Proof Reference
$\varepsilon_s(t)$	<code>epsilon_s</code> <code>symbolic_entropy()</code>	/ §3 Eq.(1); Lemma 1; Lemma 5
$\sigma_{\text{res}}(t)$	<code>sigma_res =</code> <code>stdev(phase_hist)</code>	ε_s numerator
$d_{\text{score}}(t)$	<code>d_score: 1.0 if</code> <code>prev==5 & phi==8</code>	Lemma 5 — zero-lag skip detector
$\delta\Phi(t)$	<code>delta_phi =</code> <code> phi_act-phi_exp /9</code>	ε_s numerator; §1 notation
Φ_{act}	<code>phi_act = (depth%9)+1</code>	Actual phase; Definition 1
Φ_{exp}	<code>phi_exp =</code> <code>(prev_phase%9)+1</code>	Expected phase; Definition 1
$L_{\text{loc}}(t)$	<code>L_local</code> <code>local_curvature()</code>	/ §3 Eq.(3); Lemma 2
$\theta_w(t)$	<code>theta_w</code> <code>adaptive_threshold()</code>	/ $0.4/(1 + \gamma L)$; Lemma 2
$\theta_{\text{hard}} = 0.8$	<code>theta_hard = 0.8</code>	Hard ceiling; Lemma 3
$\chi(t)$	<code>chi_t / chi_t_override()</code>	§3 Eq.(2); Theorem 1
β	$\beta_{\text{impl}}/f_{144} \geq \ln 5$	Proposition 6
ψ_1	<code>psil_trace</code> <code>BaselineCapture</code>	/ Assumption A1
$v(t)$	<code>v_score</code> <code>PhaseVelocityAccumulator</code>	/ §9 OQ3 (engineering ext.)
$\varrho(t)$	<code>recursion_n</code>	Recursion depth; Lemma 1
ϱ_{min}	<code>worst case = 1</code>	Proposition 6; Lemma 4
P_{max}	<code>max numerator change per</code> <code>step</code>	Lemma 1 — bounded attack rate

C3. Part I — Utility Functions and Data Structures

The following functions and classes provide building blocks used throughout the monitor. They carry no direct proof role but underpin every metric computed in `symbolic_entropy()` and `score()`.

Module header and version history

Proof reference: *Consolidation history corresponds to proof versions v4.1–v4.3. §9 documents all additions by version.*

Listing 1: Module docstring — consolidation history and proof guarantee summary

```

1  """
2  memory/drift.py
3  ProtoForge / RSOC -- Core Drift Detection Module
4  AI.Web Inc. | Nicholas Jacob Bogaert
5
6  Consolidates:
7      - Full drift monitor implementation (ProtoForge v2.0)
8      - Adaptive  $\chi(t)$  threshold additions (chi_convergence_proof, March
9        2026)
10         . local_curvature()          -- second-derivative curvature from
11           phase_hist
12         . adaptive_threshold()       --  $\theta_w(L) = 0.4 / (1 +$ 
13           gamma.L_local)
14         . score() wiring             -- replaces hardcoded 0.8 with
15           adaptive  $\theta_w$ 
16      - Phase velocity accumulator v4.2 (rsoc_proof_v4.3, S9 OQ3,
17        engineering extension)
18         . PhaseVelocityAccumulator   -- detects acceleration attacks
19           (Phi5->6->7->8)
20         . v(t) = 1 when N_v=3 sequential steps complete in < 18.7 ms
21         . Fourth detection path in score() after d_score hard-fire
22      - Baseline integrity v4.2 (rsoc_proof_v4.2, S9 Assumption A1)
23         . BaselineCapture            -- multi-point consensus psil
24           capture (N>=3)
25         . HMAC anchor verification   -- session-level integrity check
26           before measurement
27         . Converts Assumption A1 from implicit precondition to verifiable
28           property
29
30 Proof guarantee (v4.3):  $\beta > P_{\max}^{\theta_w} / \rho_{\min}$  is satisfied
31 analytically:
32  $\beta_{\text{impl}/f_{144}} \geq \ln(5) \approx 1.609 > 1/2 = P_{\max}^{\theta_w} / \rho_{\min}$ 
33 (Proposition 1).
34 A100 benchmarking is confirmatory validation, not logical
35 prerequisite.
36 Assumption A1 hardening via BaselineCapture; see rsoc_proof_v4.3.pdf
37 S9.
38 Note: v(t) / PhaseVelocityAccumulator is an engineering extension;
39 formal proof
40 of Theorem 1 covers the single-step adversary only (see S9 OQ3).
41 """

```

fft() Pure-Python DFT fallback (stdlib-only, no numpy)

Used by `DriftTaxonomy.detect_oscillation()` to identify resonant drift. Ap-

pears only in the taxonomy path, not in the ε_s pipeline.

`_shannon_norm()` *Normalized Shannon entropy of payload string*

$H/H_{\max}$ for the payload JSON string. Contributes H_{norm} (weight $\alpha = 0.30$) to the composite drift score in `score()`. Not part of ε_s directly.

Listing 2: `fft()` and `_shannon_norm()` — structural entropy helpers

```

1 # Pure Python DFT fallback (stdlib-only, no numpy)
2 #
3
4 def fft(data: List[float]) -> List[complex]:
5     """Pure Python DFT fallback for FFT -- slower but stdlib-only."""
6     n = len(data)
7     if n <= 1:
8         return data
9     result = []
10    for k in range(n):
11        sum_val = 0j
12        for t in range(n):
13            angle = -2j * math.pi * t * k / n
14            sum_val += data[t] * math.exp(angle)
15        result.append(sum_val)
16    return result
17
18
19 #
20 # Structural / entropy helpers
21 #
22
23 def _shannon_norm(p: str) -> float:
24     """Normalized Shannon entropy of a string."""
25     n = len(p)
26     if n < 2:
27         return 0.0
28     counts = Counter(p)
29     H = 0.0
30     for count in counts.values():
31         probability = count / n
32         if probability > 0:
33             H -= probability * math.log2(probability)
34     unique_chars = len(counts)
35     H_max = math.log2(min(unique_chars, n))
36     return H / H_max if H_max > 0 else 0.0

```

`shape_signature()` / `_canonicalize()` / `_jaccard()` / helpers

`shape_signature()` extracts a depth/type-count signature from nested JSON. `_canonicalize()` strips volatile fields (timestamps, UUIDs, agent IDs) for stable MinHash comparison. `_jaccard()` and `_type_mismatch()` feed `enhanced_chi_struct()`.

Listing 3: Structural helpers — shape signature, canonicalization, Jaccard distance

```

1 def shape_signature(obj: Any) -> Tuple[str, int, List[int]]:
2     """Extract structural signature from nested data."""
3     max_depth = 0
4     type_counts = [0] * 6 # dict, list, str, num, bool, null
5     key_list = []
6
7     def traverse(node, depth=0):
8         nonlocal max_depth
9         max_depth = max(max_depth, depth)
10        if isinstance(node, dict):
11            type_counts[0] += 1
12            for key, value in node.items():
13                key_list.append(key)
14                traverse(value, depth + 1)
15        elif isinstance(node, list):
16            type_counts[1] += 1
17            for item in node:
18                traverse(item, depth + 1)
19        elif isinstance(node, str):
20            type_counts[2] += 1
21        elif isinstance(node, (int, float)):
22            type_counts[3] += 1
23        elif isinstance(node, bool):
24            type_counts[4] += 1
25        elif node is None:
26            type_counts[5] += 1
27
28    traverse(obj)
29    key_signature =
30        hashlib.md5(''.join(sorted(key_list)).encode()).hexdigest()[:8]
31    return key_signature, max_depth, type_counts
32
33 def _canonicalize(obj: Any) -> Any:
34     """Remove volatile fields and normalize structure."""
35     if isinstance(obj, dict):
36         result = {}
37         for k, v in sorted(obj.items()):
38             if _volatile_key(k):
39                 continue
40             result[k] = _canonicalize(v)
41         return result
42     elif isinstance(obj, list):
43         return [_canonicalize(x) for x in obj]
44     elif isinstance(obj, str):
45         if UUID_RE.match(obj):
46             return "<UUID>"

```

```

47         return obj
48     else:
49         return obj
50
51
52 def _volatile_key(k: str) -> bool:
53     """Identify keys that change every call but don't indicate drift."""
54     lk = k.lower()
55     return any(x in lk for x in ['agent_id', 'ts', 'timestamp',
56                                 'prev_hash', 'uuid', '_id'])
57
58 def _json_str(x: Any) -> str:
59     """Stable JSON serialization."""
60     return json.dumps(x, sort_keys=True, separators=(',', ':'))
61
62
63 def _type_tag(v: Any) -> str:
64     """Get type name for value."""
65     if v is None:
66         return "null"
67     t = type(v).__name__
68     if t == 'bool':
69         return "bool"
70     if t in ('int', 'float'):
71         return "num"
72     if t == 'str':
73         return "str"
74     if t == 'list':
75         return "arr"
76     if t == 'dict':
77         return "obj"
78     return "other"
79
80
81 def _jaccard(a: set, b: set) -> float:
82     """Jaccard distance between sets."""
83     if not a and not b:
84         return 0.0
85     intersection = len(a & b)
86     union = len(a | b)
87     return 1.0 - (intersection / union) if union > 0 else 1.0
88
89
90 def _type_mismatch(base: dict, now: dict) -> float:
91     """Calculate type mismatch rate."""
92     if not base:
93         return 0.0
94     mismatches = sum(1 for k, t in base.items() if k in now and now[k]
95                     != t)
96     return mismatches / max(1, len(base))

```

WeightedMinHash *Similarity hashing — ψ_1 comparison backbone*

Proof reference: *Remark 1 (Notation $\leftrightarrow$ Code).* `minhash.similarity(h1, psil_trace)` returns θ_τ . $\sigma_{\text{res}} = \text{stdev}(\{1 - \theta_\tau\})$.

Partitions hash budget across payload categories (core_state 50%, auxiliary 30%, metadata 20%). `similarity()` returns the fraction of matching hash slots — this is θ_τ in the proof. The entry stored in `phase_hist` for step τ is $(1 - \theta_\tau)$; σ_{res} is the standard deviation of that history.

Listing 4: WeightedMinHash — ψ_1 comparison; θ_τ computation

```

1
2 class WeightedMinHash:
3     def __init__(
4         self,
5         weights={'core_state': 0.5, 'auxiliary': 0.3, 'metadata': 0.2},
6         total_hashes=32
7     ):
8         self.weights = weights
9         self.total_hashes = total_hashes
10        self.alloc = {}
11        cumulative = 0
12        for k, w in sorted(weights.items()):
13            count = int(w * total_hashes)
14            self.alloc[k] = (cumulative, count)
15            cumulative += count
16        self.primes = [2**i % 2**16 for i in range(total_hashes)]
17
18    def hash(self, s: str, category='core_state') -> List[str]:
19        if category not in self.alloc:
20            return []
21        start, count = self.alloc[category]
22        if count == 0:
23            return []
24        hashes = []
25        for i in range(start, start + count):
26            h = hashlib.sha256((s +
27                               str(self.primes[i])).encode()).hexdigest()[:8]
28            hashes.append(h)
29        return hashes
30
31    def similarity(self, h1: List[str], h2: List[str]) -> float:
32        if not h1 or not h2:
33            return 0.0
34        return sum(a == b for a, b in zip(h1, h2)) / len(h1)
35
36 #
37 # EMA (Exponential Moving Average)
38 #

```


EMA *Exponential Moving Average — crossing prediction*

Tracks the composite drift score and exposes `predict_crossing( $\theta_{\text{hard}}$ )`, returning estimated steps until the score reaches 0.8. Feeds the `WARN_PREDICTIVE` action when `crossing < 5` steps.

Listing 5: EMA — drift trajectory prediction for early warning

```

1
2 class EMA:
3     def __init__(self, alpha=0.2):
4         self.alpha = alpha
5         self.value = None
6
7     def update(self, x: float) -> 'EMA':
8         if self.value is None:
9             self.value = x
10        else:
11            self.value = self.alpha * x + (1 - self.alpha) * self.value
12        return self
13
14    def predict_crossing(self, tau: float, epsilon=1e-6) -> float:
15        if self.value is None or abs(self.value - tau) < epsilon:
16            return float('inf')
17        if self.value < tau:
18            return float('inf') # Not crossing upward
19        return max(0, math.log((tau - self.value) / (self.value - tau))
20                  / math.log(1 - self.alpha))
21
22 #
23 # DriftTaxonomy
24 #

```

DriftTaxonomy *Drift classification — catastrophic / evolutionary / resonant / semantic*

Classifies the drift `entropy_hist` series into five types using step-change detection, trend analysis, and FFT oscillation detection. Only *catastrophic* and *evolutionary* types interact with $\chi(t)$ in the primary action path.

Listing 6: DriftTaxonomy — five-class drift classifier

```

1
2 class DriftTaxonomy:
3     def __init__(self):
4         self.semantic_threshold = 0.5
5         self.structural_threshold = 0.5

```

```

6
7 def detect_step_change(self, series: List[float]) -> bool:
8     if len(series) < 2:
9         return False
10    return abs(series[-1] - series[-2]) /
        (statistics.stdev(series[:-1]) + 1e-6) > 3
11
12 def detect_trend(self, series: List[float]) -> bool:
13     if len(series) < 5:
14         return False
15    return sum(b - a for a, b in zip(series, series[1:])) > 0
16
17 def detect_oscillation(self, series: List[float]) -> bool:
18     if len(series) < 8:
19         return False
20    spec = fft(series)
21    freqs = [abs(f) for f in spec]
22    dom_freq = max(range(len(freqs)), key=lambda i: freqs[i]) /
        len(series)
23    return 0.3 <= dom_freq < 0.7
24
25 def classify(self, entropy_series: List[float], structural_changes:
float, semantic_dist: float):
26     if self.detect_step_change(entropy_series):
27         return "catastrophic", self.emergency_response
28     elif self.detect_trend(entropy_series):
29         return "evolutionary", self.guided_correction
30     elif self.detect_oscillation(entropy_series):
31         return "resonant", self.damping_response
32     elif semantic_dist > self.semantic_threshold:
33         return "semantic", self.semantic_realignment
34     elif structural_changes > self.structural_threshold:
35         return "structural", self.structural_normalization
36     return "benign", lambda x: None
37
38 def emergency_response(self, agent):
39     if agent:
40         agent.state['blocked'] = True
41
42 def guided_correction(self, agent):
43     if agent:
44         agent.state['budget'] = min(agent.state.get('budget', 0.3),
0.3)
45
46 def damping_response(self, agent):
47     if agent:
48         agent.state['residue'] *= 0.7
49
50 def semantic_realignment(self, agent):
51     if agent:
52         agent.state['psi_trace'] = agent.psil_trace if
hasattr(agent, 'psil_trace') else []

```

```

53
54     def structural_normalization(self, agent):
55         if agent and hasattr(agent, 'task_type'):
56             agent.state['schema'] =
57                 self.schema_types.get(agent.task_type, {})
58
59 #
60 # -----
61 # DriftCircuitBreaker
62 # -----

```

DriftCircuitBreaker *Resilience layer — trips on sustained exceedances*

Standard three-state circuit breaker (closed/open/half-open) on the composite drift score. Trips after `max_failures=3` consecutive exceedances of τ_{hard} . Recovery timeout: 60s. This is a system resilience mechanism; it does not alter ε_s measurement or $\chi(t)$ convergence.

Listing 7: DriftCircuitBreaker — sustained-failure resilience

```

1
2 class DriftCircuitBreaker:
3     def __init__(self, max_failures=3, recovery_timeout=60):
4         self.state = "closed"
5         self.failure_count = 0
6         self.max_failures = max_failures
7         self.recovery_timeout = recovery_timeout
8         self.trip_time = 0
9
10    def handle(self, drift_score: float, threshold: float):
11        if self.state == "closed":
12            if drift_score > threshold:
13                self.failure_count += 1
14                if self.failure_count > self.max_failures:
15                    self.state = "open"
16                    self.trip_time = time.time()
17                    return "circuit tripped"
18            else:
19                self.failure_count = max(0, self.failure_count - 1)
20                return "closed"
21        elif self.state == "open":
22            if time.time() - self.trip_time > self.recovery_timeout:
23                self.state = "half_open"
24                return "testing recovery"
25        elif self.state == "half_open":
26            if drift_score < threshold:
27                self.state = "closed"
28                self.failure_count = 0
29                return "recovered"

```

```

30         else:
31             self.state = "open"
32             return "recovery_failed"
33     return self.state
34
35
36 #
37 # -----
38 # HierarchicalMonitor stack
39 #
40 # -----

```

HierarchicalMonitor / SwarmLevelMonitor / ClusterLevelMonitor

Three-level monitoring stack. `SwarmLevelMonitor` detects global anomalies across agents via median absolute deviation ($|e - \text{median}| / \text{MAD} > 3$). `ClusterLevelMonitor` isolates the highest-variance task cluster. `CONSENSUS_CORRECT` fires in `check_and_firewall()` when `swarm_z > 3`.

Listing 8: HierarchicalMonitor — swarm / cluster / agent detection stack

```

1
2 class SwarmLevelMonitor:
3     def detect_anomaly(self, agents):
4         eps = [a.current_entropy for a in agents if hasattr(a,
5             'current_entropy')]
6         if not eps:
7             return False
8         median = statistics.median(eps)
9         mad = statistics.median([abs(e - median) for e in eps]) or 1e-6
10        return any(abs(e - median) / mad > 3 for e in eps)
11
12 class ClusterLevelMonitor:
13     def locate_drift(self, agents):
14         clusters = defaultdict(list)
15         for a in agents:
16             if hasattr(a, 'task_type') and hasattr(a,
17                 'current_entropy'):
18                 cluster_key = (a.task_type, int(a.current_entropy * 10))
19                 clusters[cluster_key].append(a)
20         max_variance = 0
21         problem_cluster = None
22         for key, cluster in clusters.items():
23             if len(cluster) > 1:
24                 entropies = [a.current_entropy for a in cluster]
25                 variance = statistics.variance(entropies) if
26                     len(entropies) > 1 else 0
27                 if variance > max_variance:
28                     max_variance = variance
29                     problem_cluster = key

```

```

28         return problem_cluster
29
30
31 class AgentLevelMonitor:
32     def diagnose(self, cluster_id, cmd, payload):
33         return "diagnosed_cluster", cluster_id
34
35
36 class HierarchicalMonitor:
37     def __init__(self):
38         self.swarm = SwarmLevelMonitor()
39         self.cluster = ClusterLevelMonitor()
40         self.agent = AgentLevelMonitor()
41
42     def check(self, all_agents, cmd, payload):
43         if not all_agents:
44             return "benign", 0.0
45         if self.swarm.detect_anomaly(all_agents):
46             cluster_id = self.cluster.locate_drift(all_agents)
47             return self.agent.diagnose(cluster_id, cmd, payload)
48         return "benign", 0.0
49
50
51 #
52 # -----
53 # PhaseVelocityAccumulator
54 # -----
55 #
56 # Implements the phase velocity accumulator  $v(t)$  defined in RSOC Proof
57 # v4.1,
58 # S9 Known Limitation / OQ3.
59 #
60 # Purpose: detect multi-step acceleration attacks that evade  $d\_score$  by
61 # executing  $\Phi_5 \rightarrow \Phi_6 \rightarrow \Phi_7 \rightarrow \Phi_8$  across three individually-legal
62 # steps rather than
63 # one illegal  $\Phi_5 \rightarrow \Phi_8$  skip. Each individual step has  $\Phi_{act} ==$ 
64 #  $\Phi_{exp}$ , so
65 #  $d\_score = 0$  and  $\Delta\Phi = 0$  at every step. The attack is only
66 # visible as
67 # anomalous  $\ast velocity \ast$  -- three sequential +1 steps completing in less
68 # time
69 # than three normal 144 Hz cycles require.
70 #
71 # Parameters (from proof S9, OQ3):
72 #  $N_v = 3$  -- minimum consecutive sequential steps to flag
73 #  $\tau_{min\_ms} = 18.7\text{ ms}$  -- ( $0.1 \times \tau \ast = 0.1 \times 187\text{ ms}$ )
74 # Normal 3-step minimum:  $3 \times (1000/144) \approx$ 
75 # 20.8 ms
76 # Acceleration attack completes 3 steps in <
77 # 18.7 ms
78 #

```

```

71 # v(t) = 1 when >= N_v sequential steps (each with Phi_act == Phi_exp)
    # are
72 #     observed within tau_min_ms milliseconds.
73 # v(t) = 0 otherwise.
74 #
    =====
75 # ADDITION v4.2 -- BaselineCapture (Assumption A1 hardening)
76 #
77 # Implements the two-part hardening described in RSOC Proof v4.2 S9 /
    # A1 Remark:
78 #     (i) Multi-point consensus: psil = MedianHash over N>=3 cold-start
        # captures.
79 #     An adversary must corrupt a strict majority of capture points
        # to shift psil.
80 #     (ii) External anchor: HMAC(psil, key) stored separately; session
        # verifies
81 #     match before any chi(t) measurement is trusted.
82 #
83 # Without this class, psil_trace is a single-point capture (A1 holds
        # but is
84 # unverifiable). With this class, A1 can be checked at session start.
85 #
    =====

```

C4. Part II — Assumption A1: BaselineCapture

Formal status: Assumption A1 (Trusted Initialization) — $T_{\text{adv}} > T_{\text{cap}}$

Hardened via $N \geq 3$ consensus capture + HMAC anchor in an external trust domain.

Without BaselineCapture: ψ_1 is a single-point capture — A1 holds but is unverifiable at session start.

With BaselineCapture: A1 can be checked before any $\chi(t)$ measurement is trusted. An adversary must corrupt a strict majority of $N \geq 3$ capture points to shift ψ_1 .

BaselineCapture *Multi-point consensus ψ_1 capture with HMAC integrity anchor*

Proof reference: §9 Assumption A1 Remark: “hardened via $N \geq 3$ consensus capture and external HMAC anchor (BaselineCapture).”

`capture()` produces ψ_1 as the median-hash across $n_{\text{points}} \geq 3$ sequential cold-start payloads. The median is the hash with the lowest summed pairwise Jaccard distance to all others. A pairwise distance sanity check (`avg_dist > 0.5` raises `RuntimeError`) validates that the environment is clean before binding. The HMAC anchor binds ψ_1 to an externally-stored secret; `verify()` raises if they diverge, signalling post-capture corruption.

Recommended usage (from §9 A1 Remark in proof):

```
cap  = BaselineCapture(monitor, hmac_key=b"system-secret")
psil = cap.capture(agent, n_points=3)    # consensus median-hash
anchor = cap.anchor                      # store in separate trust
    domain
# ... session begins ...
cap.verify(anchor)                      # raises RuntimeError if
    corrupted
cap.bind_agent(agent)                   # sets agent.psil_trace = psil
```

Listing 9: BaselineCapture — Assumption A1 hardening (multi-point consensus + HMAC anchor)

```
1
2 class BaselineCapture:
3     """
4         Multi-point consensus baseline capture with HMAC integrity anchor.
5
6         Implements Assumption A1 hardening from RSOC Proof v4.2, S9.
7
8         Usage
9         -----
10        cap = BaselineCapture(monitor, hmac_key=b"system-secret")
11        psil = cap.capture(agent, n_points=3)    # consensus hash
12        anchor = cap.anchor                    # store externally
13        ...
14        cap.verify(anchor)                    # raises if
15        corrupted
16        agent.psil_trace = cap.psil            # bind to agent
17
18        Parameters
19        -----
20        monitor : DriftMonitor
21            Live monitor whose minhash / _canonicalize methods are used.
22        hmac_key : bytes
23            System secret for HMAC anchor. Store in a separate trust domain
24            (env-var, HSM, TPM) -- never co-located with psil itself.
25        """
26
27        MIN_POINTS: int = 3
28
29        def __init__(self, monitor, hmac_key: bytes = b"rsoc-default-key"):
30            self._monitor = monitor
31            self._hmac_key = hmac_key
32            self.psil: str | None = None
33            self.anchor: str | None = None
34            self._capture_hashes: list[str] = []
35
36        # -----
37        def capture(self, agent, payload_sequence: list | None = None,
38            n_points: int = 3) -> str:
```

```

39         Capture psil as the median-hash over n_points sequential
           payloads.
40
41     Parameters
42     -----
43     agent : object
44         Agent whose state is being measured. Must have 'state'
           dict.
45     payload_sequence : list, optional
46         Pre-supplied sequence of payloads (for testing). If None,
47         n_points copies of agent.state are used (cold-start
           assumption:
48         the agent has not yet been influenced by an adversary).
49     n_points : int
50         Number of capture points. Must be  $\geq$  MIN_POINTS (3).
51
52     Returns
53     -----
54     str
55         The consensus psil hash. Also sets self.psil and
           self.anchor.
56
57     Raises
58     -----
59     ValueError
60         If n_points < MIN_POINTS.
61     RuntimeError
62         If the capture hashes show unexpected spread (> 0.5 avg
           pairwise
63         distance), signalling a non-clean environment.
64     """
65     if n_points < self.MIN_POINTS:
66         raise ValueError(
67             f"n_points={n_points} < MIN_POINTS={self.MIN_POINTS}. "
68             "Theorem A1 requires  $N \geq 3$  for majority-corruption
               resistance."
69         )
70
71     mh = self._monitor.minhash
72     if payload_sequence is None:
73         # Cold-start: use n_points snapshots of agent's current
           state.
74         payload_sequence = [dict(agent.state) for _ in
                               range(n_points)]
75
76     hashes = []
77     for payload in payload_sequence[:n_points]:
78         can = _canonicalize(payload)
79         h = mh.hash(_json_str(can), 'core_state')
80         hashes.append(h)
81
82     # Sanity check: pairwise distances should be low for clean

```



```

        cold-start.
83     distances = []
84     for i in range(len(hashes)):
85         for j in range(i + 1, len(hashes)):
86             distances.append(1.0 - mh.similarity(hashes[i],
87                                                  hashes[j]))
87     if distances:
88         avg_dist = sum(distances) / len(distances)
89         if avg_dist > 0.5:
90             raise RuntimeError(
91                 f"BaselineCapture: avg pairwise hash distance
92                 {avg_dist:.3f} > 0.5. "
93                 "Environment may not be clean. Verify cold-start
94                 conditions before "
95                 "binding psil."
96             )
97
98     # Median hash = hash with lowest summed distance to all others.
99     best_hash = min(hashes, key=lambda h: sum(
100         1.0 - mh.similarity(h, other) for other in hashes
101     ))
102
103     self._capture_hashes = hashes
104     self.psil = best_hash
105
106     # Compute HMAC anchor.
107     import hmac as _hmac
108     import hashlib
109     h_anchor = _hmac.new(
110         self._hmac_key,
111         self.psil.encode() if isinstance(self.psil, str) else
112         str(self.psil).encode(),
113         hashlib.sha256
114     ).hexdigest()
115     self.anchor = h_anchor
116
117     return self.psil
118
119 # -----
120 def verify(self, stored_anchor: str) -> bool:
121     """
122     Verify that the current psil matches a stored HMAC anchor.
123
124     Call at the start of each measurement session. A mismatch means
125     psil was modified after initial capture (Assumption A1
126     violated).
127
128     Parameters
129     -----
130     stored_anchor : str
131         The anchor produced by a prior 'capture()' call, stored in a
132         separate trust domain.

```

```

129
130     Returns
131     -----
132     bool
133         True if psil is intact.
134
135     Raises
136     -----
137     RuntimeError
138         If psil is None (capture() not yet called) or if HMAC
139         mismatch.
140     """
141     if self.psil is None:
142         raise RuntimeError("BaselineCapture.verify(): capture() has
143                             not been called.")
144
145     import hmac as _hmac
146     import hashlib
147     expected = _hmac.new(
148         self._hmac_key,
149         self.psil.encode() if isinstance(self.psil, str) else
150         str(self.psil).encode(),
151         hashlib.sha256
152     ).hexdigest()
153
154     if not _hmac.compare_digest(expected, stored_anchor):
155         raise RuntimeError(
156             "BaselineCapture.verify(): HMAC mismatch. "
157             "psil may have been corrupted after initial capture. "
158             "Assumption A1 (Trusted Initialization) cannot be
159             confirmed. "
160             "Abort measurement session."
161         )
162     return True
163
164 # -----
165 def bind_agent(self, agent) -> None:
166     """Assign self.psil to agent.psil_trace after successful
167     capture."""
168     if self.psil is None:
169         raise RuntimeError("call capture() before bind_agent()")
170     agent.psil_trace = self.psil
171
172 #
173
174 # Note: v(t) is per-command, not global, since each agent maintains its
175 # own phase sequence. The accumulator is stored in
176 DriftMonitor.pva_bufs.

```

C5. Part III — OQ3 Engineering Extension: PhaseVelocityAccumulator

Formal status: Engineering extension — NOT covered by Theorem 1

Theorem 1 covers the single-step $S_5 \rightarrow S_8$ adversary. The multi-step acceleration attack ($S_5 \rightarrow S_6 \rightarrow S_7 \rightarrow S_8$, each step individually valid with $\Phi_{\text{act}} = \Phi_{\text{exp}}$) evades d_{score} because no individual step triggers the categorical predicate.

$v(t) = 1$ when $N_v = 3$ consecutive sequential steps complete in $< \tau_{\min} = 18.7$ ms.

Normal 3-step minimum: $3 \times (1000/144) \approx 20.8$ ms. $\tau_{\min} = 0.1 \times \tau^* = 0.1 \times 187$ ms.

Formal proof of $v(t)$ correctness: deferred (§9 OQ3).

PhaseVelocityAccumulator $v(t) \in \{0, 1\}$ — acceleration attack detector

Proof reference: §9 OQ3: “multi-step acceleration variant addressed by $v(t)$ in `drift.py`; formal proof deferred.”

`update()` records each step as a `(timestamp_ms, is_sequential)` tuple in a circular buffer of capacity $N_v + 1 = 4$. After $N_v = 3$ entries: (a) all N_v steps must be sequential, and (b) elapsed time $< \tau_{\min} = 18.7$ ms. `elapsed=0` (mock environments) is treated as acceleration. `reset()` clears the buffer after $\chi(t)$ fires.

Listing 10: PhaseVelocityAccumulator — $v(t)$, OQ3 acceleration-attack engineering extension

```

1
2 class PhaseVelocityAccumulator:
3     """
4     Tracks recent sequential phase steps and fires  $v(t)=1$  when  $N_v$ 
5     consecutive legal steps complete in under tau_min_ms milliseconds.
6
7     Each entry in the buffer is a (timestamp_ms, is_sequential) tuple.
8     is_sequential = True iff Phi_act == Phi_exp for that step.
9
10    Usage::
11
12        pva = PhaseVelocityAccumulator()
13        v = pva.update(  $\Phi_{\text{act}}$  ,  $\Phi_{\text{exp}}$  , time.monotonic() * 1000)
14        if v == 1:
15            # acceleration attack detected
16    """
17
18    N_V: int = 3 # minimum sequential steps
19    TAU_MIN_MS: float = 18.7 # maximum wall-time for  $N_V$  steps to be
20    flagged
21
22    def __init__(self):

```

```

22     # Circular buffer: stores (timestamp_ms, is_sequential) pairs.
23     # Capacity N_V+1 is sufficient -- we only look back N_V steps.
24     self._buf: deque = deque(maxlen=self.N_V + 1)
25
26     def update(self,     $\Phi_{act}$  : int,     $\Phi_{exp}$  : int, now_ms: float) -> int:
27         """
28         Record one step and return the current velocity score  $v(t)$  ?
29         {0, 1}.
30
31         Parameters
32         -----
33          $\Phi_{act}$  : int
34             Actual phase computed from payload (1--9).
35          $\Phi_{exp}$  : int
36             Expected phase ((prev_phase % 9) + 1).
37         now_ms : float
38             Current wall-clock time in milliseconds (monotonic).
39
40         Returns
41         -----
42         int
43             1 if an acceleration attack is detected, 0 otherwise.
44         """
45         is_seq = (  $\Phi_{act}$  ==  $\Phi_{exp}$  )
46         self._buf.append((now_ms, is_seq))
47
48         # Need at least N_V entries to make a determination.
49         if len(self._buf) < self.N_V:
50             return 0
51
52         # Check the N_V most recent entries.
53         recent = list(self._buf)[-self.N_V:]
54
55         # All N_V steps must be sequential (each  $\Phi_{act}$  ==  $\Phi_{exp}$ ).
56         if not all(s for _, s in recent):
57             return 0
58
59         # Time span of those N_V steps must be under tau_min_ms.
60         t_start = recent[0][0]
61         t_end    = recent[-1][0]
62         elapsed = t_end - t_start
63
64         # elapsed == 0 can occur in test/mock environments where
65         # timestamps
66         # are identical; treat as acceleration (infinitely fast).
67         if elapsed < self.TAU_MIN_MS:
68             return 1
69
70         return 0
71
72     def reset(self):
73         """Clear accumulated history (e.g., after chi(t) fires)."""

```

```

72         self._buf.clear()
73
74
75 #
76 # -----
77 # DriftTrace
78 # -----

```

C6. Part IV — DriftMonitor: The Core Class

DriftMonitor is the central class of the module. It owns all rolling statistics, the Min-Hash instance, taxonomy, circuit breaker, hierarchical monitor, EMA predictor, and one PhaseVelocityAccumulator per command key (pva_bufs). The primary external interface is `check_and_firewall()`; all proof-relevant computation flows through `score() → symbolic_entropy() → adaptive_threshold() → chi_t_override()`.

Listing 11: `DriftMonitor.__init__()` — class initialisation and parameter declarations

```

1
2 class DriftMonitor:
3     def __init__(self, echo_path: Path):
4         self.echo_path = echo_path
5         self.cmd_stats = defaultdict(_RollingStats)
6         self.schema_keys = defaultdict(set)
7         self.schema_types = defaultdict(dict)
8         self.last_hashes = deque(maxlen=256)
9         self.drift_log = Path('storage/logs/drift.log')
10        self.drift_log.parent.mkdir(parents=True, exist_ok=True)
11        self.minhash = WeightedMinHash()
12        self.taxonomy = DriftTaxonomy()
13        self.circuit_breaker = DriftCircuitBreaker()
14        self.hierarchical = HierarchicalMonitor()
15        self.predictor = EMA()
16        self.trace = DriftTrace()
17        self.entropy_hist = deque(maxlen=64)
18        self.budget = 0.3
19        self.checkpoints = []
20        # Per-command PhaseVelocityAccumulators (one per cmd key).
21        # Implements v(t) from RSOC Proof v4.1, S9 OQ3.
22        self.pva_bufs: dict = defaultdict(PhaseVelocityAccumulator)
23        # Composite drift weights
24        self.alpha = 0.3 # H_norm weight
25        self.beta = 0.15 # S_norm weight
26        self.gamma = 0.1 # novelty weight
27        self.delta = 0.25 # chi_struct weight
28        self.epsilon = 0.1 # A_time weight
29        self.ε_s = 0.1 # symbolic entropy weight

```

```

30     self.tau_soft = 0.5
31     self.tau_hard = 0.7
32     self.setup_logging()
33     self.load_hashes()
34
35     # -----
36     # Setup / persistence
37     # -----
38
39     def setup_logging(self):
40         self.logger = logging.getLogger('drift_monitor')
41         self.logger.setLevel(logging.INFO)
42         handler = RotatingFileHandler(
43             self.drift_log, maxBytes=10 * 1024 * 1024, backupCount=5
44         )
45         self.logger.addHandler(handler)
46
47     def load_hashes(self):
48         if self.echo_path.exists():
49             with open(self.echo_path, 'r') as f:
50                 for line in f.readlines()[-256:]:
51                     try:
52                         rec = json.loads(line)
53                         can = _canonicalize(rec.get("payload", {}))
54                         self.last_hashes.append(
55                             self.minhash.hash(_json_str(can),
56                                             'core_state')
57                         )

```

C7. Part V — The ε_s Pipeline

symbolic_entropy() $\varepsilon_s(t)$ — *Structural Drift Metric*

Proof reference: §3 Eq.(1): $\varepsilon_s = (\sigma_{\text{res}} + d_{\text{score}} + \delta\Phi) / \max(1, \varrho)$. Lemma 1 (bounded attack rate). Lemma 5 (zero-lag detection). Remark (operational definition of ε_s).

Computes all three numerator components of ε_s in a single call. Φ_{act} is derived from the payload depth via `shape_signature()`. $\Phi_{\text{exp}} = (\text{prev_phase} \% 9) + 1$ (Definition 1). $\delta\Phi = |\Phi_{\text{act}} - \Phi_{\text{exp}}|/9$.

On d_{score} : It is a *deterministic categorical predicate*, not a statistical estimator. It fires iff `prev_phase` = 5 and $\Phi_{\text{act}} = 8$. Both values are computed from the current payload in the current call: d_{score} fires at $t = 0$ of the skip with zero causal lag, independent of the `phase_hist` window. This is the property that makes it resistant to distribution-matching evasion: there is no distribution to match.

ε_s is **not** clipped to $[0, 1]$: at $\varrho = 1$, $d_{\text{score}} = 1$, $\delta\Phi = 8/9$, $\sigma_{\text{res}} = 0.5$ gives $\varepsilon_{s,\text{max}} \approx 2.44$.

Listing 12: `symbolic_entropy()` — $\varepsilon_s(t)$, Φ_{act} , d_{score} in one call

```

1
2 def symbolic_entropy(self, cmd, payload, recursion_n, prev_phase,
3   psil_trace):
4     """
5     ?_s = (?_res + d_score + deltaPhi) / max(1, recursion_n)
6
7     ?_res    -- resonance variance (stdev of phase_hist)
8     d_score  -- Luciferian skip detector: 1.0 if Phi5->8 else 0.0
9     deltaPhi -- phase deviation, normalized [0,1]
10    rho      -- recursion depth (denominator); deeper = more stable
11
12    d_score is a deterministic binary flag, not a statistical
13    accumulator.
14    Both prev_phase and Phi_act are computed in this call, so
15    d_score fires
16    at t=0 of the skip with zero lag -- independent of phase_hist
17    window.
18    This is the basis of Lemma 5 (one-step simultaneous detection).
19    """
20    can = _canonicalize(payload)
21    _, depth, type_vec = shape_signature(can)
22    Phi_act = (depth % 9) + 1
23    if any(danger in str(can).lower() for danger in ['replicate',
24      'spawn', 'infinite']):
25        Phi_act = min(9, Phi_act + 3)
26    Phi_exp = (prev_phase % 9) + 1
27     $\delta \Phi$  = abs(Phi_act - Phi_exp) / 9.0
28
29    # d_score: deterministic zero-lag Luciferian skip fingerprint.
30    # Fires iff prev_phase==5 and Phi_act==8 -- the exact Phi5->8
31    # attack pattern.
32    # This is NOT a statistical estimate; it is a categorical
33    # ground truth flag.
34    d_score = 1.0 if prev_phase == 5 and Phi_act == 8 else 0.0
35
36    h1 = self.minhash.hash(_json_str(can), 'core_state')
37    theta_t = 1 - self.minhash.similarity(h1, psil_trace) if
38    psil_trace else 0.0
39
40    stats = self.cmd_stats[cmd]
41    stats.phase_hist.append(1 - theta_t)
42     $\sigma_{res}$  = statistics.stdev(stats.phase_hist) if
43    len(stats.phase_hist) > 1 else 0.0
44
45     $\varepsilon_s$  = (  $\sigma_{res}$  + d_score +  $\delta \Phi$  ) / max(1,
46    recursion_n)
47    #  $\sigma_{res}$  in [0,0.5] (stdev of 1-theta_t values in [0,1]).
48    #  $\delta \Phi$  in [0, 8/9] (max |  $\Phi_{act}$  -  $\Phi_{exp}$  | = 8 for phi in
49    # {1..9}).
50    # eps_s NOT clipped. At rho=1: d_score=1,  $\delta \Phi$  =2/9,
51     $\sigma_{res}$  =0.5 gives eps_s=31/18 max.

```

```

40     return εs, Phi_act, d_score # d_score returned for
    hard-fire logic
41
42     # -----
43     # ADDITION 1 -- local_curvature()
44     # -----

```

local_curvature() $L_{\text{loc}}(t)$ — Second-Order Phase Variance

Proof reference: §3 Eq.(3): $L_{\text{loc}} = \text{Var}(\{\Phi_{\text{hist}}(t-i+1) - \Phi_{\text{hist}}(t-i)\}_{i=1}^{k-1})$. Lemma 2 (curvature inversion under explicit baseline).

Variance of consecutive differences in the most recent window entries of phase_hist. Normal sequential +1 progression gives approximately constant diffs ($L_{\text{loc}} \approx 0$). A skip inserts a deviation spike visible as large second-order variance even when absolute values look unremarkable. Window default = 8, matching the short-window assumption of Lemma 2.

Listing 13: `local_curvature()` — $L_{\text{loc}}(t)$, second-derivative curvature from `phase_hist`

```

1  def local_curvature(self, cmd: str, window: int = 8) -> float:
2      """
3      Compute local curvature from recent phase_hist variance.
4      High curvature = rapid phase transitions = Luciferian skip
5      territory.
6
7      Uses a short window (default 8) so detection responds within
8      the same recursion depth as the attack.
9
10     Returns L_local: float in [0, ?) where:
11         0.0 = flat, stable, no curvature
12         0.5 = moderate drift, warning zone
13         1.0+ = high curvature, Luciferian skip likely
14
15     Implementation uses second derivative (variance of consecutive
16     differences) rather than first derivative, because the
17     Luciferian
18     skip produces a spike in rate-of-change-of-rate-of-change --
19     visible
20     in L_local even when absolute phase values look normal.
21     """
22     stats = self.cmd_stats[cmd]
23     hist = list(stats.phase_hist)
24     if len(hist) < 2:
25         return 0.0
26     recent = hist[-window:] if len(hist) >= window else hist
27     if len(recent) < 2:
28         return 0.0

```



```

27     diffs = [abs(recent[i + 1] - recent[i]) for i in
28               range(len(recent) - 1)]
29     return statistics.variance(diffs) if len(diffs) > 1 else
30         (diffs[0] if diffs else 0.0)
31
32     # -----
33     # ADDITION 2 -- adaptive_threshold()
34     # -----

```

adaptive_threshold() $\theta_w(t) = 0.4/(1 + \gamma \cdot L_{\text{loc}})$

Proof reference: §3 Eq.(3): $\theta_w(t) = 0.4/(1 + \gamma \cdot L_{\text{loc}}(t))$. Lemma 2: the attack that most threatens the system simultaneously lowers the threshold that catches it.

θ_w drops monotonically as L_{loc} rises. At stable operation ($L = 0$): $\theta_w = 0.40$. As curvature increases during a skip: θ_w falls — the attacker sharpens the detector against itself. $\theta_w \in (0, 0.4]$ for all $L \geq 0$. $\theta_{\text{hard}} = 0.8$ is the absolute ceiling and is never raised.

γ tuning: start at 0.5. Lower toward 0.3 if false positives occur on legitimate rapid transitions. Raise toward 0.8 if skips pass under high load. Use DriftTrace logs to calibrate.

Listing 14: adaptive_threshold() — $\theta_w(t)$ computation; curvature inversion

```

1  def adaptive_threshold(self, cmd: str, gamma: float = 0.5) -> float:
2      """
3           $\theta_w(L) = 0.4 / (1 + \gamma \cdot L_{\text{local}})$ 
4
5          As local curvature  $L$  increases, the firing threshold drops.
6           $\chi(t)$  fires earlier in exactly the regions the Luciferian skip
7          exploits. The attacker cannot raise their attack rate without
8          simultaneously lowering the threshold that catches them.
9
10         Threshold values by curvature (gamma=0.5):
11             L = 0.0 (stable):       $\theta_w = 0.400$ 
12             L = 0.5 (moderate):     $\theta_w = 0.267$ 
13             L = 1.0 (high):         $\theta_w = 0.200$ 
14             L = 2.0 (severe skip):  $\theta_w = 0.133$ 
15
16         gamma tuning:
17             Start at 0.5. Lower toward 0.3 if false positives on
18             legitimate
19             rapid phase transitions (e.g., bootstrapping acceleration).
20             Raise toward 0.8 if Luciferian skips get through under high
21             load.
22             Use DriftTrace logs to calibrate.
23
24         Proof reference: Lemma 2 (chi_convergence_proof.pdf) -- the
25         attack
26         that most threatens the system also most strongly triggers early

```

```

25         detection.  $\theta_w(L)$  cancels the curvature advantage the
                attacker gains.
26         """
27         L = self.local_curvature(cmd)
28         return 0.4 / (1.0 + gamma * L)
29
30         # -----
31         # chi(t) override
32         # -----

```

C8. Part VI — $\chi(t)$ Override and Detection Logic

chi_t_override() *Grace override — rebind, fold residue*

Proof reference: §3 Eq.(2): $d\chi(t)/dt = -\beta \cdot \chi(t) + \nabla\psi_1 \cdot \Theta(t)$. Theorem 1: $\chi(t)$ converges before ε_s reaches θ_{hard} .

Called when any detection path fires. Computes $\chi_t = \nabla\psi_1 \times \theta_\tau$, folds the residue into `agent.state['residue']`, and classifies the response curve as *convergent* (`diff < 0.2`) or *oscillating*. Convergent curves set `agent.phase = 9` and produce the RESURRECT action in `check_and_firewall()`.

Listing 15: `chi_t_override()` — $\chi(t)$ correction; convergent/oscillating classification

```

1
2  def chi_t_override(self, agent, payload,  $\varepsilon_s$ ):
3      """
4      Grace override: rebinds agent to psil_trace, folds residue,
5      checks curve (convergent / oscillating).
6
7      Called when ?_s crosses  $\theta_w$  (adaptive) or  $\theta_{\text{hard}}$  (0.8).
8      """
9      if not hasattr(agent, 'psil_trace'):
10         return "no_baseline", 0.0
11     psil_trace = agent.psil_trace
12     can = _canonicalize(payload)
13     h1 = self.minhash.hash(_json_str(can), 'core_state')
14     grad_psi = 0.5 # Real implementation: derive from state diffs
15     theta_t = 1 - self.minhash.similarity(h1, psil_trace)
16
17     # chi_t is non-zero whenever called (threshold decision lives
18     # in score/check_and_firewall)
19     chi_t = grad_psi * theta_t
20
21     residue =  $\varepsilon_s$  * 0.3
22     current_residue = agent.state.get('residue', 0.0)
23     agent.state['residue'] = (current_residue + residue) / 2

```

```

24     diff = abs(grad_psi - theta_t)
25     curve = "convergent" if diff < 0.2 else "oscillating"
26     if curve == "convergent":
27         agent.phase = 9
28         self.trace.log(f"chi(t)={chi_t:.3f}, curve={curve}",      εs ,
29                     agent.phase)
29     return curve, chi_t
30
31     # -----
32     # ADDITION 3 -- score() with adaptive threshold wiring
33     # -----

```

score() *Composite drift + four-path detection logic*

Proof reference: §5 Complete Detection Rule Eq.(4): three-path priority: (1) d_{score} hard-fire, (2) $\varepsilon_s > \theta_w$ adaptive, (3) $\varepsilon_s > \theta_{\text{hard}}$ ceiling. Implementation adds v_{score} as path (2a). Lemmas 1–5 each map to one decision branch.

`score()` is the heart of the monitor. It computes all drift components, runs `PhaseVelocityAccumula` for the current step, derives θ_w and L_{loc} , then executes the four detection paths in strict priority order:

1. $d_{\text{score}} = 1 \Rightarrow$ HARD-FIRE $\chi(t)$ unconditionally [Lemma 5]. Categorical predicate; zero causal lag; bypasses θ_w entirely.
2. $v_{\text{score}} = 1 \Rightarrow$ ACCELERATION ATTACK [OQ3]. $N_v = 3$ sequential steps in $< \tau_{\text{min}} = 18.7$ ms. Engineering extension; not covered by Theorem 1.
3. $\varepsilon_s > \theta_w(t) \Rightarrow$ ADAPTIVE PATH [Lemma 2]. θ_w tightens as curvature rises.
4. $\varepsilon_s > \theta_{\text{hard}} \Rightarrow$ HARD CEILING (backstop) [Lemma 3]. `elif` – not re-evaluated if path 3 already fired. Catches slow-build attacks.

Listing 16: `score()` — composite drift computation and four-path detection priority logic

```

1
2     def score(self, cmd, payload, phase_latency_ms, recursion_n,
3               prev_phase,
4               psil_trace, all_agents, agent=None):
5         """
6         Compute composite drift score and component details.
7
8         ADAPTIVE THRESHOLD CHANGE (chi_convergence_proof.pdf):
9         After computing ?_s, we derive  θw via
10            adaptive_threshold(cmd).
11            θw replaces the old implicit 0.8 ceiling as the *warning*
12            trigger.
13            θhard = 0.8 remains the absolute ceiling.

```

```

12     If  $\theta_w > \theta_w$  and an agent is provided,  $\chi(t)$  fires
        immediately
13     with a trace log showing  $\epsilon_s$ ,  $\theta_w$ , and  $L_{\text{local}}$ .
14     """
15     can = _canonicalize(payload)
16     pstr = _json_str(can)
17     H_norm = _shannon_norm(pstr)
18
19     stats = self.cmd_stats[cmd]
20     median, mad = stats.size_params()
21     S_norm = _size_norm(len(pstr), median, mad)
22
23     h1 = self.minhash.hash(pstr, 'core_state')
24     N = 0.2 if h1 not in self.last_hashes else 0.0
25
26     chi_struct, aux = self._structure_features(cmd, payload)
27     chi_struct = self.enhanced_chi_struct(cmd, aux, can)
28
29     A_time = min(1.0, max(0, (phase_latency_ms - 500) / 500))
30
31      $\epsilon_s$ , Phi_act, d_score = self.symbolic_entropy(
32         cmd, payload, recursion_n, prev_phase, psil_trace
33     )
34
35     # ?? PHASE VELOCITY ACCUMULATOR  $v(t)$  (Proof v4.1, S9 OQ3) ??????
36     # Records this step and returns 1 if  $N_V=3$  consecutive
        sequential
37     # steps complete in under  $\tau_{\text{min\_ms}}=18.7$  ms -- the acceleration
38     # attack signature. Computed before the detection branch so
        that
39     # the velocity flag is available alongside d_score.
40     now_ms = time.monotonic() * 1000.0
41     Phi_exp_for_pva = (prev_phase % 9) + 1
42     v_score = self.pva_bufs[cmd].update(Phi_act, Phi_exp_for_pva,
        now_ms)
43     # ?? END PHASE VELOCITY ACCUMULATOR
        ?????????????????????????????????
44
45     # ?? ADAPTIVE THRESHOLD (Addition 3)
        ?????????????????????????????????
46     #  $\theta_w$  drops as local curvature rises; attacker sharpens
        detector
47     # against itself.  $\theta_{\text{hard}}$  is the absolute ceiling -- never
        raised.
48      $\theta_w$  = self.adaptive_threshold(cmd) # dynamic warning floor
49      $\theta_{\text{hard}}$  = 0.8 # hard ceiling,
        always 0.8
50
51     chi_fired = False
52     chi_result = None
53     L_local = self.local_curvature(cmd)
54

```

```

55     # ?? LEMMA 5: d_score HARD-FIRE
56     # d_score is a deterministic binary flag, not a statistical
57     # measure.
58     # It fires at t=0 of the Luciferian skip (Phi5->8) by
59     # construction --
60     # zero lag, independent of phase_hist window and
61     # local_curvature().
62     # This resolves the open one-step lag problem from
63     # chi_convergence_proof.pdf:
64     # local_curvature() may lag one step at window boundaries, but
65     # d_score
66     # fires simultaneously with the skip in the same
67     # symbolic_entropy() call.
68     #
69     # When d_score == 1.0, chi(t) fires unconditionally --
70     # bypassing  $\theta_w$ 
71     # entirely. The adaptive threshold still governs all other
72     # drift paths.
73     #
74     # Proof: d_score = 1 iff prev_phase==5 and Phi_act==8. Both
75     # values are
76     # computed from the current payload in the current call.
77     # Therefore
78     # d_score fires at the recursion step of the skip, not one step
79     # later.
80     # This is sufficient to guarantee chi(t) convergence at all
81     # recursion
82     # depths (including recursion_n >= 4 where  $\epsilon_s < \theta_w$  from
83     # d_score+deltaPhi
84     # alone), because the categorical signal bypasses the threshold
85     # entirely.
86     # ?? END LEMMA 5
87     #
88     # Zero-lag Luciferian skip hard-fire -- no threshold check
89     # needed.
90     # Lemma 5: d_score is categorical, fires at t=0 of the skip.
91     self.trace.log(
92         f"LUCIFERIAN_SKIP_HARDFIRE: d_score=1.0 at
93         recursion_n={recursion_n}"
94         f",  $\epsilon_s$  = { $\epsilon_s$ :.3f},  $\theta_w$  = { $\theta_w$ :.3f}
95         (bypassed)",
96          $\epsilon_s$ ,
97         Phi_act
98     )
99     if agent is not None:
100         chi_result = self.chi_t_override(agent, payload,
101              $\epsilon_s$  )
102         chi_fired = True

```

```

87         self.pva_bufs[cmd].reset()
88     elif v_score == 1:
89         # Phase velocity accumulator: acceleration attack detected.
90         # Three consecutive sequential (+1) steps completed in under
91         # tau_min_ms (18.7 ms), indicating compressed traversal that
92         # evades d_score. Proof v4.1 S9 OQ3 Known Limitation.
93         self.trace.log(
94             f"ACCELERATION_ATTACK: v(t)=1 at
95               recursion_n={recursion_n}"
96             f", eps={    εs  :.3f} (3 sequential steps in <18.7
97               ms)",
98             εs ,
99             Phi_act
100         )
101         if agent is not None:
102             chi_result = self.chi_t_override(agent, payload,
103               εs )
104             chi_fired = True
105             self.pva_bufs[cmd].reset()
106     elif εs > θw :
107         # Standard adaptive threshold path (Lemma 2).
108         self.trace.log(
109             f"chi_t fired (adaptive): eps={    εs  :.3f} >    θw
110               ={    θw  :.3f}"
111             f" (L={L_local:.3f})",
112             εs ,
113             Phi_act
114         )
115         if agent is not None:
116             chi_result = self.chi_t_override(agent, payload,
117               εs )
118             chi_fired = True
119     elif εs > θhard :
120         # Hard ceiling path: catches slow-build attacks that stay
121         # below θw
122         # but eventually accumulate past 0.8. θhard is NOT the
123         # primary
124         # detector -- it is the backstop. elif is correct (not if):
125         # if adaptive
126         # already fired, this path is skipped.
127         self.trace.log(
128             f"chi_t fired (hard ceiling): eps={    εs  :.3f} >
129               θhard ={    θhard  }",
130             εs ,
131             Phi_act
132         )
133         if agent is not None:
134             chi_result = self.chi_t_override(agent, payload,
135               εs )
136             chi_fired = True
137     # ?? END DETECTION PATHS
138     ?????????????????????????????????????????????????????????

```

```

128
129     drift_in = (
130         self.alpha * H_norm
131         + self.beta * S_norm
132         + self.gamma * N
133         + self.delta * chi_struct
134         + self.epsilon * A_time
135         + self.       $\epsilon_s$       *       $\epsilon_s$ 
136     )
137     self.entropy_hist.append(drift_in)
138
139     semantic_dist = (
140         1 - self.minhash.similarity(h1, psil_trace) if psil_trace
141         else 0.0
142     )
143     drift_type, response = self.taxonomy.classify(
144         list(self.entropy_hist), chi_struct, semantic_dist
145     )
146
147     crossing =
148         self.predictor.update(drift_in).predict_crossing(       $\theta_{hard}$  )
149
150     swarm_eps = [a.current_entropy for a in all_agents if
151         hasattr(a, 'current_entropy')]
152     self.swarm_median = statistics.median(swarm_eps) if swarm_eps
153     else drift_in
154     swarm_mad = statistics.median([abs(e - self.swarm_median) for e
155         in swarm_eps]) or 1e-6
156     z = abs(drift_in - self.swarm_median) / swarm_mad
157
158     details = {
159         "H_norm": H_norm,
160         "S_norm": S_norm,
161         "N": N,
162         "chi_struct": chi_struct,
163         "A_time": A_time,
164         "       $\epsilon_s$       ":       $\epsilon_s$       ,
165         "d_score": d_score,                # exposed: 1.0 = Luciferian
166         "v_score": v_score,                # 1 = acceleration attack
167         "       $\theta_w$       ":       $\theta_w$       ,
168         "       $\theta_{hard}$       ":       $\theta_{hard}$       ,
169         "L_local": L_local,
170         "chi_fired": chi_fired,
171         "chi_result": chi_result,
172         "drift_type": drift_type,
173         "crossing": crossing,
174         "swarm_z": z,
175     }
176     return drift_in, details, aux, Phi_act, response

```

```

173 # -----
174 # check_and_firewall -- top-level entry point
175 # -----

```

C9. Part VII — Top-Level Entry Point

check_and_firewall() *Full firewall check — action resolution*

Proof reference: §8 *Detection Path by Recursion Depth*. Crossover depth $q^* = \lfloor \varepsilon_{s,\max}/\theta_w \rfloor$; for $q > q^*$, Lemma 5 is the only guaranteed detection path.

The single external interface. Pulls `recursion_n`, `prev_phase`, and `psil_trace` from the agent (defaults 1, 0, [] if absent), calls `score()`, then resolves the final action in priority order:

```

circuit_tripped / recovery_failed ⇒ BLOCK
recovered / testing_recovery ⇒ ALLOW
crossing < 5 (EMA) ⇒ WARN_PREDICTIVE
drift ≥  $\tau_{\text{hard}}$  ⇒ BLOCK
chi_fired + convergent ⇒ RESURRECT
swarm_z > 3 ⇒ CONSENSUS_CORRECT
evolutionary + budget ⇒ checkpoint → ROLLBACK

```

Listing 17: `check_and_firewall()` — top-level firewall; action resolution; logging

```

1
2  def check_and_firewall(self, cmd, payload, agent=None,
3                          phase_latency_ms=0.0,
4                          all_agents=None):
5      """
6      Full firewall check. Pass agent for chi(t) override eligibility.
7      all_agents enables swarm consensus correction.
8      """
9      try:
10         all_agents = all_agents or []
11         recursion_n = getattr(agent, 'recursion_n', 1) if agent
12         else 1
13         prev_phase = getattr(agent, 'prev_phase', 0) if agent else 0
14         psil_trace = getattr(agent, 'psil_trace', []) if agent else
15         []
16
17         drift_in, details, aux, Phi_act, response = self.score(
18             cmd, payload, phase_latency_ms, recursion_n, prev_phase,
19             psil_trace, all_agents, agent=agent
20         )
21
22         stats = self.cmd_stats[cmd]

```



```

20         tau = stats.drift_params()
21
22         action = self.circuit_breaker.handle(drift_in,
23                                             self.tau_hard)
24         if action in ("circuit_tripped", "recovery_failed"):
25             action = "BLOCK"
26         elif action in ("recovered", "testing_recovery"):
27             action = "ALLOW"
28         elif details['crossing'] < 5:
29             action = "WARN_PREDICTIVE"
30         elif drift_in >= self.tau_hard:
31             action = "BLOCK"
32
33         # chi(t) may have already fired inside score() via adaptive
34         # threshold;
35         # check_and_firewall respects that result if it produced a
36         # convergent curve.
37         if details.get('chi_fired') and details.get('chi_result'):
38             curve, chi_t = details['chi_result']
39             if curve == "convergent":
40                 action = "RESURRECT"
41         elif agent:
42             # Fallback: also fire at hard ceiling if adaptive
43             # didn't catch it
44             if drift_in >= details[' $\theta_{hard}$ ']:
45                 curve, chi_t = self.chi_t_override(agent, payload,
46                                                    details[' $\epsilon_s$ '])
47                 if curve == "convergent":
48                     action = "RESURRECT"
49         elif drift_in >= tau:
50             action = "WARN"
51         if agent:
52             response(agent)
53
54         if details['drift_type'] == "evolutionary" and
55         details[' $\epsilon_s$ ] < self.budget:
56             self.checkpoints.append(agent.state.copy() if agent
57                                     else {})
58             self.budget -= details[' $\epsilon_s$ ']
59         elif self.checkpoints:
60             if agent:
61                 agent.state = self.checkpoints.pop()
62             action = "ROLLBACK"
63
64         if details['swarm_z'] > 3:
65             action = "CONSENSUS_CORRECT"
66             if agent:
67                 agent.current_entropy = self.swarm_median
68
69         if action in ("ALLOW", "WARN", "RESURRECT"):
70             stats.sizes.append(len(aux['pstr']))
71             stats.scores.append(drift_in)

```

```

65         self.schema_keys[cmd] |= aux['now_keys']
66         self.schema_types[cmd].update(aux['now_types'])
67         self.last_hashes.append(self.minhash.hash(aux['pstr'],
68             'core_state'))
69
70     self.trace.log(action, drift_in, Phi_act)
71     self.logger.info(json.dumps({
72         "ts": time.time(),
73         "cmd": cmd,
74         "action": action,
75         "drift": round(drift_in, 4),
76         "tau": round(tau, 4),
77         "θw": round(details['θw'], 4),
78         "Llocal": round(details['Llocal'], 4),
79         "components": {k: v for k, v in details.items()
80             if k not in ('chi_result',)},
81     }))
82
83     if action.startswith("WARN"):
84         print(
85             f"WARNING: drift={drift_in:.3f} >= tau={tau:.3f}, "
86             f"type={details['drift_type']}, "
87             f"θw={details['θw']:.3f}, "
88             f"L={details['Llocal']:.3f}"
89         )
90     elif action == "BLOCK":
91         print(f"DRIFT_BLOCKED: drift={drift_in:.3f} >=
92             tau_hard={self.tau_hard:.2f}")
93     elif action == "RESURRECT":
94         print(
95             f"RESURRECTED: chi(t) fired at
96             ?_s={details['εs']:.3f} "
97             f"(θw={details['θw']:.3f})"
98         )
99
100     return action, drift_in, {"tau": tau, **details}
101
102 except Exception as e:
103     self.logger.error(f"Drift check failed: {str(e)}")
104     return "ERROR_ALLOW", 0.0, {"error": str(e)}

```

C10. Detection Flow Diagram

The following schematic summarises the complete call chain from external invocation to final action, with proof references at each decision point.

Listing 18: Complete call chain with proof lemma annotations at each decision point

```

check_and_firewall(cmd, payload, agent, ...)
|

```

```

+-- score(cmd, payload, ...)
|
|   +-- symbolic_entropy()      -> eps_s,   $\Phi_{act}$ , d_score
|   [Lemma 1, Lemma 5]
|   |   +-- shape_signature()  ->  $\Phi_{act}$  from payload depth
|   |   +-- minhash.similarity -> theta_tau ; stdev ->  $\sigma_{res}$ 
|   |   +-- d_score predicate  -> 1.0 iff prev==5 and phi==8
|   |
|   +-- local_curvature()      -> L_local
|   [Lemma 2]
|   +-- adaptive_threshold()   ->  $\theta_w = 0.4 / (1 + \text{gamma} * L)$ 
|   [Eq. (3)]
|   +-- pva_bufs[cmd].update() -> v_score
|   [OQ3]
|   |
|   +-- d_score == 1.0?        -> HARD-FIRE chi(t)
|   [Lemma 5]
|   +-- v_score == 1?          -> chi(t) acceleration
|   [OQ3]
|   +-- eps_s >  $\theta_w$  ?        -> chi(t) adaptive
|   [Lemma 2]
|   +-- eps_s >  $\theta_{hard}$  ?      -> chi(t) hard ceiling
|   [Lemma 3]
|
+-- chi_t_override()           -> convergent / oscillating
|   [Theorem 1]
|   +-- grad_psi x theta_tau   -> chi(t) value
|   +-- diff < 0.2             -> RESURRECT / agent.phase=9
|
+-- circuit_breaker.handle()   -> BLOCK / ALLOW / RESURRECT
+-- EMA.predict_crossing()      -> WARN_PREDICTIVE (crossing < 5)
+-- swarm_z > 3                 -> CONSENSUS_CORRECT
+-- return action, drift_in, details

```

C11. Runtime Output Reference

`check_and_firewall()` returns (action, drift_in, details). The details dict exposes all proof-relevant quantities:

Listing 19: details dict returned by `check_and_firewall()` — proof-relevant keys

```

# Proof-relevant keys in the details dict:
details = {
    ' $\epsilon_s$ ': float, # eps_s -- structural drift metric [S3 Eq. (1)]
    'd_score': float, # 1.0 = S5->S8 phase-skip detected [Lemma 5]
    'v_score': int,   # 1 = acceleration attack detected [OQ3]
    ' $\theta_w$ ': float, # adaptive warning threshold [S3 Eq. (3)]
    ' $\theta_{hard}$ ': float, # always 0.8 [Lemma 3]
    'L_local': float, # local curvature [Lemma 2]
    'chi_fired': bool, # True if chi(t) fired this step [Theorem 1]

```

```

'chi_result': tuple, # ('convergent'|'oscillating', chi_t_value)
# -- composite score components --
'H_norm': float, # Shannon entropy (alpha = 0.30 weight)
'S_norm': float, # size deviation (beta = 0.15 weight)
'N': float, # novelty flag (gamma = 0.10 weight)
'chi_struct': float, # structural dev (delta = 0.25 weight)
'A_time': float, # latency (eps = 0.10 weight)
# -- system state --
'drift_type': str, # catastrophic / evolutionary / resonant /
    semantic / benign
'crossing': float, # EMA estimate: steps until  $\theta_{hard}$ 
'swarm_z': float, # MAD z-score vs agent swarm median
}
# JSON log entry written to storage/logs/drift.log (rotating, 10 MB):
# { ts, cmd, action, drift, tau,  $\theta_w$ , L_local, components }

```

drift.py v4.3 • AI.Web Inc. • Nicholas Jacob Bogaert • March 2026. Companion to RSOC Convergence Proof v4.8 • arXiv target: cs.NE / cs.LG.